딥러닝 개요

1 인공지능·머신러닝·딥러닝

2 퍼셉트론과 그 한계

3 다층 퍼셉트론과 역전파

4 순전파·역전파를 직접 계산하기

5 신경망의 부품

6 손실 함수

7 옵티마이저와 학습 설정

8 오늘날의 딥러닝

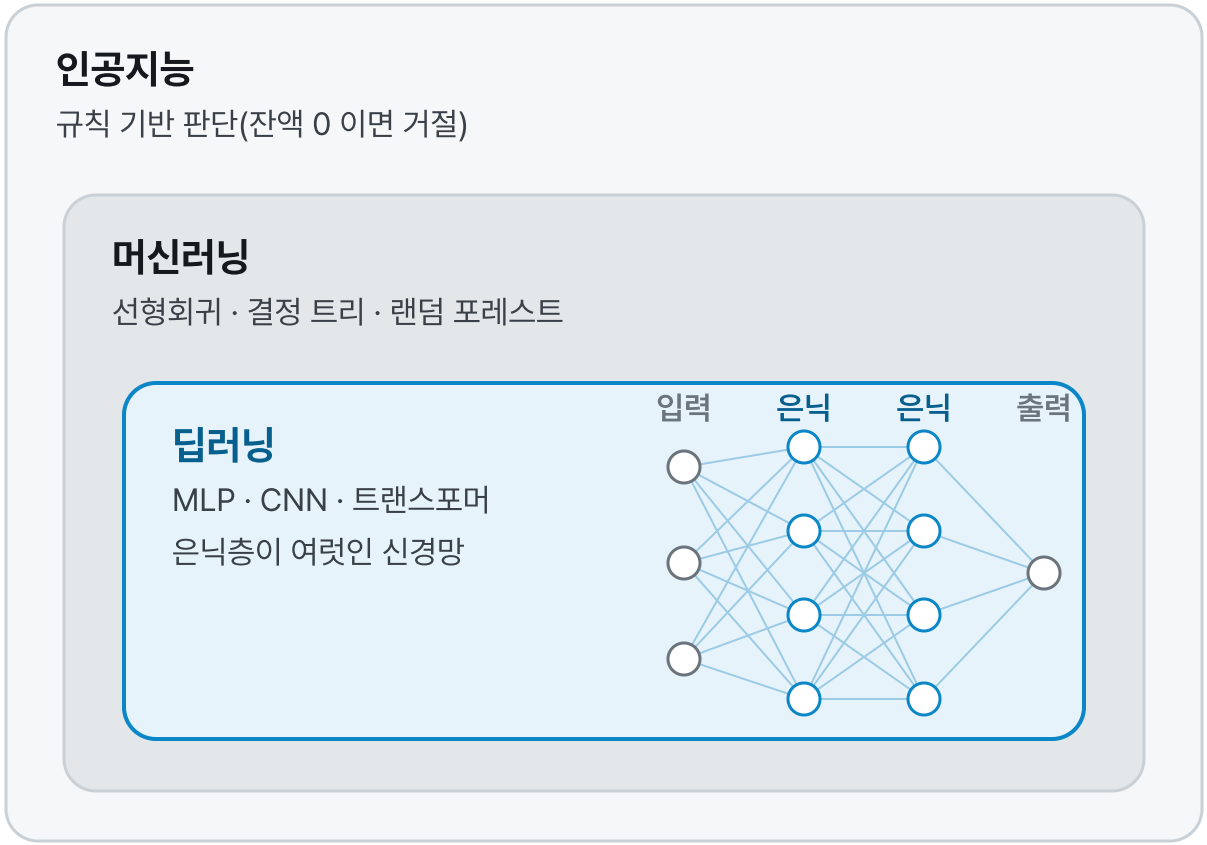
1

인공지능·머신러닝·딥러닝

딥러닝은 머신러닝과 무엇이 다른가?

세 용어의 포함 관계와, 선형회귀가 신경망 노드 하나와 같은 계산이라는 사실

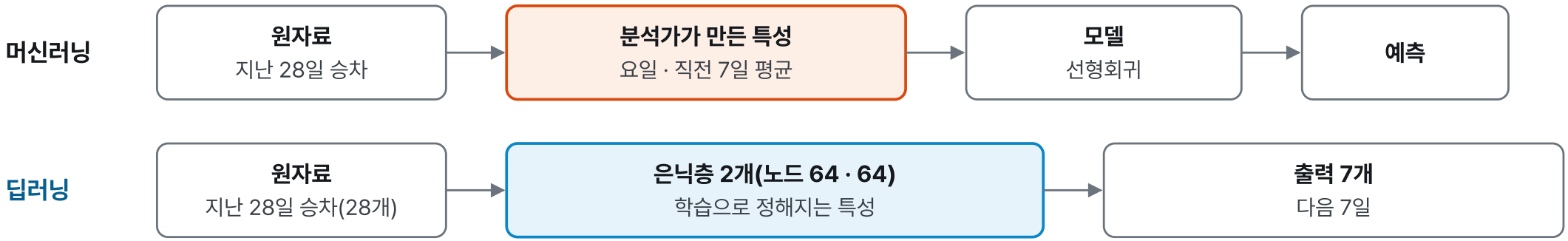
딥러닝은 머신러닝의 일부이고, 머신러닝은 인공지능의 일부다



- **인공지능**(AI, artificial intelligence): 사람이 하던 판단을 컴퓨터가 하게 하는 방법 전체. 사람이 규칙을 직접 적는 방법도 포함된다
- **머신러닝**(ML, machine learning): 규칙을 사람이 적지 않고 자료로 학습해 정하는 방법
- **딥러닝**(DL, deep learning): 은닉층이 여럿인 신경망으로 학습하는 머신러닝. **은닉층**(hidden layer)은 입력층과 출력층 사이에서 중간값을 계산하는 층이다(그림에서 「은닉」이라 적힌 두 열)

> **딥러닝은 신경망을 쓰는 여러 방법(MLP·CNN·트랜스포머 등)을 통틀어 부르는 이름이다**

머신러닝은 분석가가 만든 특성을 입력하고, 딥러닝은 원자료를 입력해 특성까지 학습으로 정한다



- **특성**(feature): 모델에 입력하는 열. 머신러닝에서는 분석가가 만든다. 예: 지하철 승차 인원을 예측할 때 만드는 「요일」·「직전 7일 평균」 열
- 딥러닝은 지난 28일의 승차 인원 28개를 그대로 입력하고, 은닉층이 계산한 값을 특성으로 쓴다. 이 값을 계산하는 가중치도 학습으로 정해진다. 딥러닝 시계열 과목의 MLP(다층 퍼셉트론: 층을 여러 개 이은 신경망, 「3. 다층 퍼셉트론과 역전파」)는 은닉층 2개에 노드가 64개씩이다
- 그래서 원자료가 이미지·글·소리·긴 계열이고 자료가 많을 때 딥러닝이 유리하다. **1만 행 안팎의 표 자료에서는 트리 모델이 여전히 더 정확한 경우가 많다**

출처: Grinsztajn, Oyallon, Varoquaux(2022), Why do tree-based models still outperform deep learning on typical tabular data?, NeurIPS 2022 Datasets and Benchmarks

선형회귀는 노드가 하나인 신경망이고, 시그모이드를 적용하면 로지스틱 회귀가 된다

노드(node) 하나는 입력에 가중치를 곱해 더하고 편향을 더한다. 선형회귀가 바로 이 계산이다.

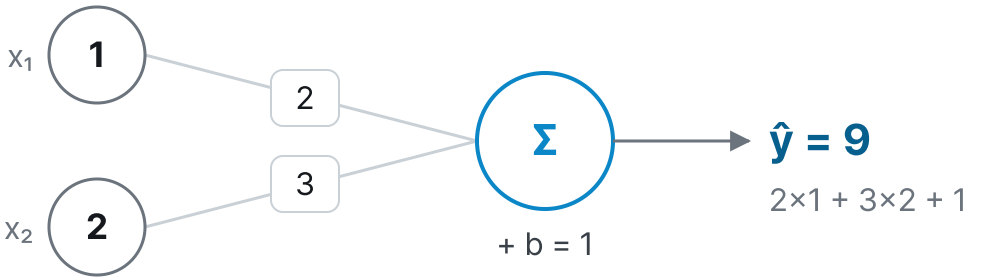
$$\hat{y} = w_1 x_1 + w_2 x_2 + b$$

기호: x_1, x_2: 입력 · w_1, w_2: 가중치 · b: 편향

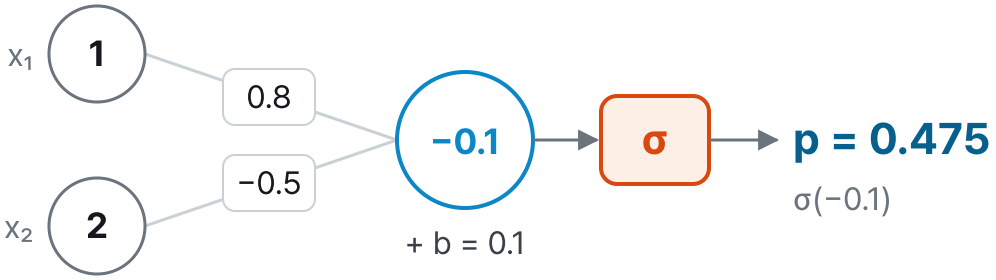
같은 노드의 출력에 **시그모이드 σ** 를 적용하면 0~1 사이의 확률이 나온다. 이것이 로지스틱 회귀다.

$$z = 0.8 \times 1 - 0.5 \times 2 + 0.1 = -0.1 \rightarrow \sigma(-0.1) = 0.475$$

선형회귀



로지스틱 회귀



> **신경망은 이 노드를 한 층에 여러 개 두고, 그 층을 여러 개 이은 것이다**

정리: 딥러닝은 신경망으로 특성까지 함께 학습하는 머신러닝이다

1 포함 관계

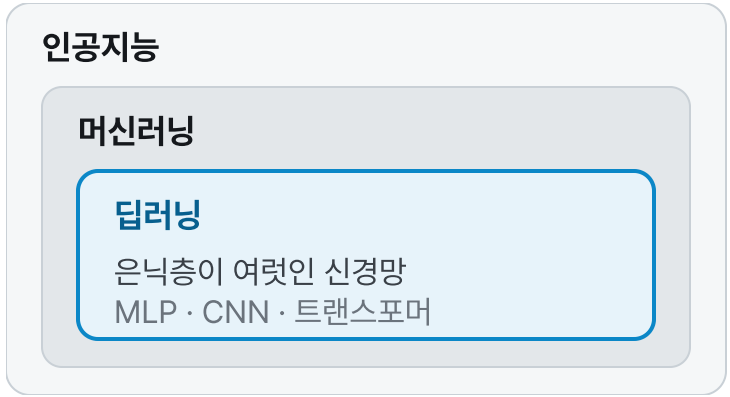
인공지능 ⊃ 머신러닝 ⊃ 딥러닝.
딥러닝은 은닉층이 여럿인 신경망으로 학습하는 방법들이다

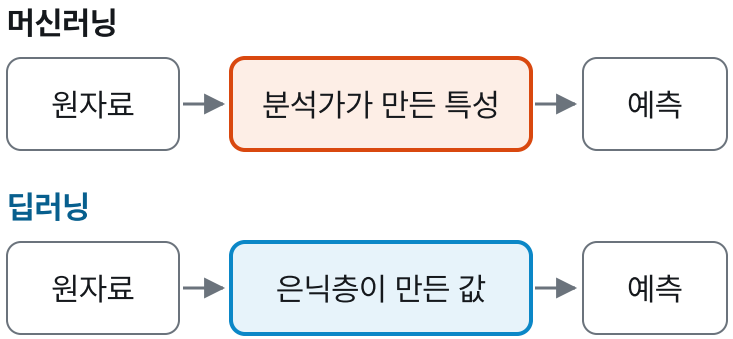
2 특성을 누가 만드나

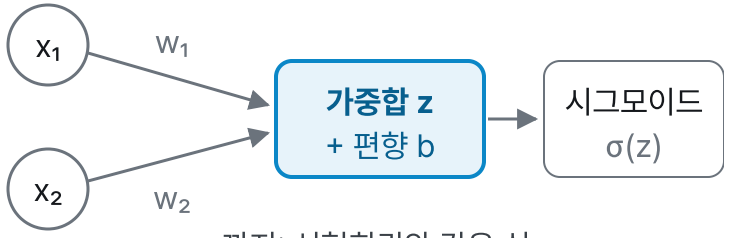
머신러닝은 분석가가 특성을 만들고, 딥러닝은 은닉층의 출력을 특성으로 쓴다. 1만 행 안팎의 표 자료는 트리 모델이 더 정확할 때가 많다

3 노드 하나

신경망의 노드 하나는 선형회귀와 같은 가중합이다. 시그모이드를 적용하면 로지스틱 회귀가 된다







z 까지: 선형회귀와 같은 식
시그모이드까지: 로지스틱 회귀

2

퍼셉트론과 그 한계

직선 하나로 점을 나누는 모델은 어디까지 풀 수 있나?

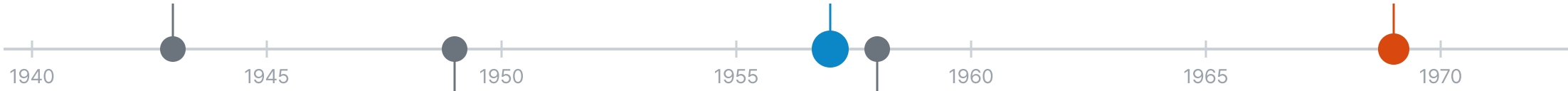
1943년 인공 뉴런 모형부터 1969년 XOR 문제까지: 퍼셉트론의 계산, 학습 규칙, 한계

신경망 연구는 1943년 인공 뉴런 모형에서 시작해 1957년 퍼셉트론으로 이어졌다

1943
매컬럭(McCulloch)·피츠(Pitts)
입력의 합이 임계값을 넘으면
1 을 출력하는 인공 뉴런 모형

1957
로젠블랫(Frank Rosenblatt)
가중치를 자료로 고치는 퍼셉트론
코넬 항공 연구소, IBM 704

1969
민스키(Minsky)·페퍼트(Papert)
『Perceptrons』: 층이 하나인
퍼셉트론의 한계를 정리



1949
헵(Donald Hebb), 심리학자
두 뉴런 사이 연결의 세기를
바꾸는 학습 규칙

1958
뉴욕 타임스 7월 8일 기사
"NEW NAVY DEVICE LEARNS BY DOING"
(해군의 새 장치, 해 보며 배운다)

- 1958년 뉴욕 타임스는 「걷고 말하고 보고 쓰는」 컴퓨터의 시작이라 보도했다. 시연에서 실제로 한 일은 **카드의 표시가 왼쪽에 있는지 오른쪽에 있는지를 50번 시도 끝에 구별한 것**이었다

출처: McCulloch·Pitts(1943) A Logical Calculus of the Ideas Immanent in Nervous Activity · Hebb(1949) The Organization of Behavior · Rosenblatt(1957) 보고서 85-460-1 · The New York Times 1958-07-08 · Cornell Chronicle 2019-09-25 · Minsky·Papert(1969) Perceptrons

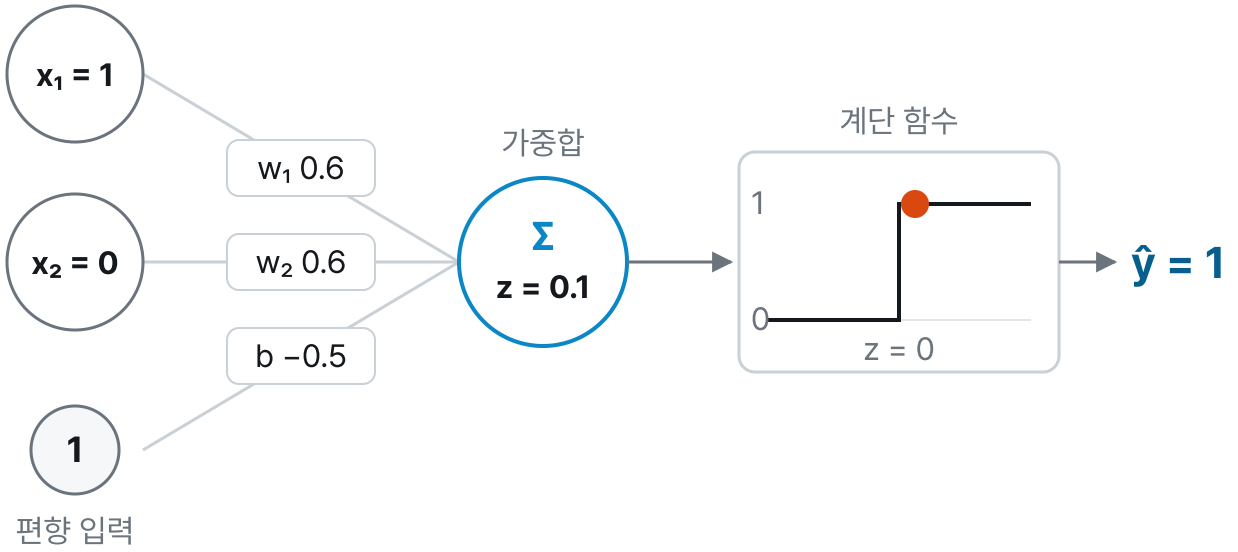
퍼셉트론은 입력에 가중치를 곱해 더하고, 그 값이 0 보다 크면 1 을 출력한다

먼저 가중합은 다음과 같다.

$$z = w_1 x_1 + w_2 x_2 + b$$

기호: x_1, x_2: 입력(0 또는 1) · w_1, w_2: 가중치 · b: 편향

다음으로 z 를 **계단 함수**(step function)에 대입해 출력을 정한다. $z > 0$ 이면 $\hat{y} = 1$, $z \leq 0$ 이면 $\hat{y} = 0$.



예: $w = (0.6,\ 0.6), b = -0.5, x = (1, 0) \rightarrow z = 0.6 - 0.5 = 0.1 > 0 \rightarrow \hat{y} = 1$

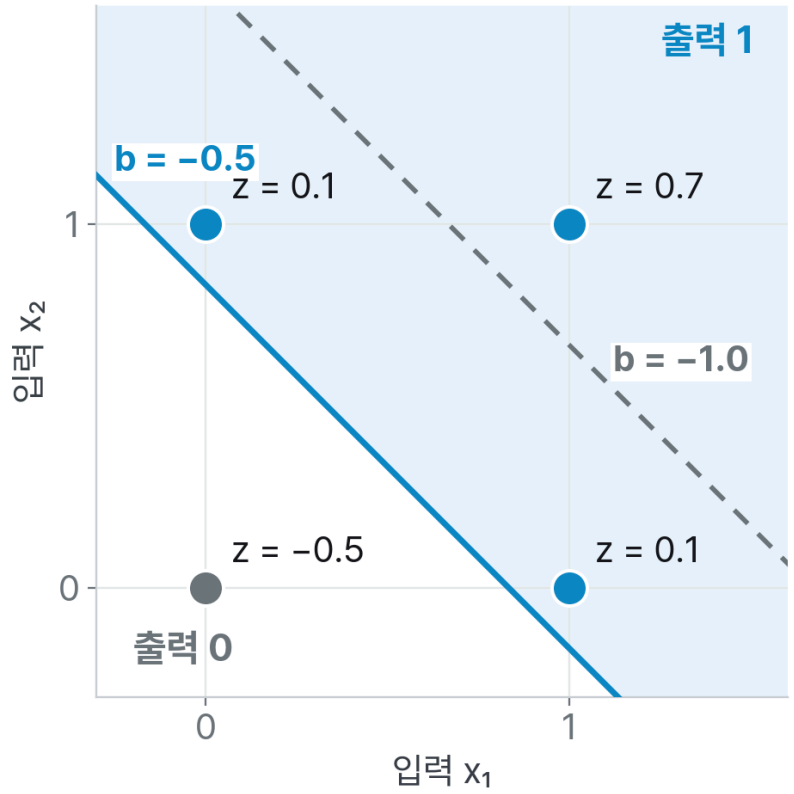
계단 함수의 유래: 1943년 매컬럭·피츠 모형이 신경 세포를 「입력의 합이 임계값을 넘으면 1」로 적었다

퍼셉트론 한 번의 계산 구조는 이렇다.

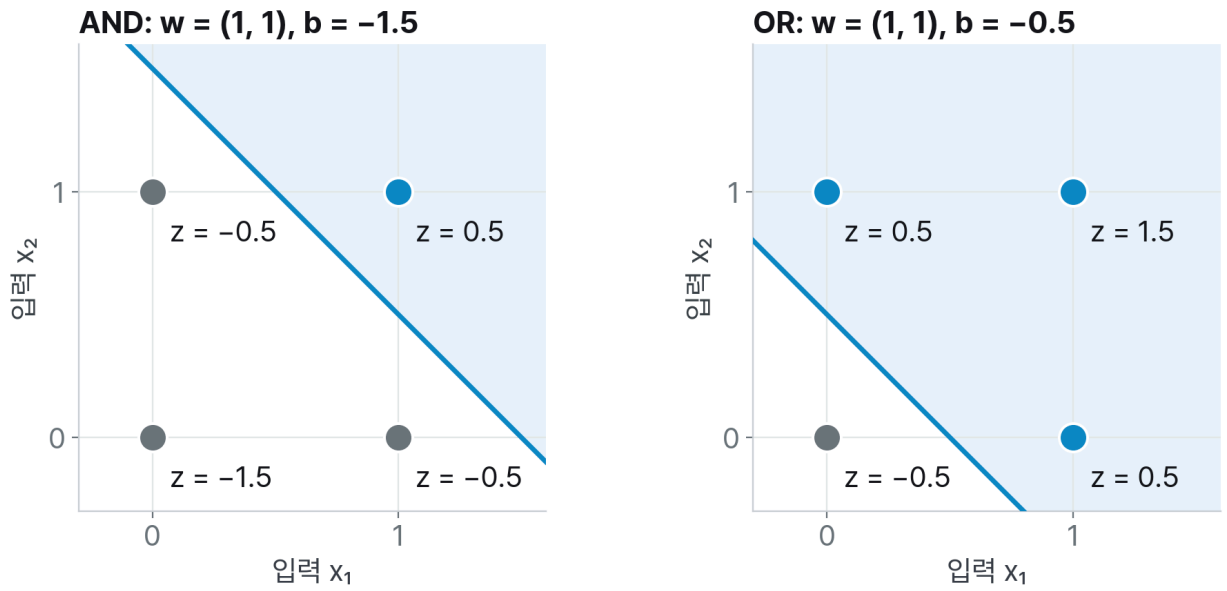
가중합 z → 계단 함수 → 0 또는 1

가중치와 편향은 점을 둘로 나누는 직선, 곧 결정 경계를 정한다

- **결정 경계**(decision boundary)는 $z = 0$ 인 점들이다.
  $w = (0.6,\ 0.6), b = -0.5$ 이면
  $0.6x_1 + 0.6x_2 - 0.5 = 0$, 곧 $x_1 + x_2 = 0.83$
- 네 점의 z: (0,0) −0.5 · (0,1) 0.1 · (1,0) 0.1 · (1,1) 0.7. 직선 위쪽 세 점이 1, 아래 한 점이 0
- 가중치는 직선의 기울기를, 편향은 직선의 위치를 바꾼다.
  $b = -1.0$ 이면 경계가 $x_1 + x_2 = 1.67$ 로 이동해 (1,1) 하나만 1 이 된다($z = 0.2$)



AND 와 OR 는 직선 하나로 나뉘고, 두 퍼셉트론은 편향만 −1.5 와 −0.5 로 다르다



x_1	x_2	AND z	AND	OR z	OR
0	0	−1.5	0	−0.5	0
0	1	−0.5	0	0.5	**1**
1	0	−0.5	0	0.5	**1**
1	1	0.5	**1**	1.5	**1**

- **AND**: 둘 다 1 일 때만 1. **OR**: 하나라도 1 이면 1. 둘 다 $w = (1, 1)$ 이고 $z = x_1 + x_2 + b$
- 경계는 AND 가 $x_1 + x_2 = 1.5$, OR 가 $x_1 + x_2 = 0.5$. **편향만 달라 기울기가 같은 직선이 평행 이동했다**

퍼셉트론 학습 규칙은 틀린 점에서만 가중치를 고친다

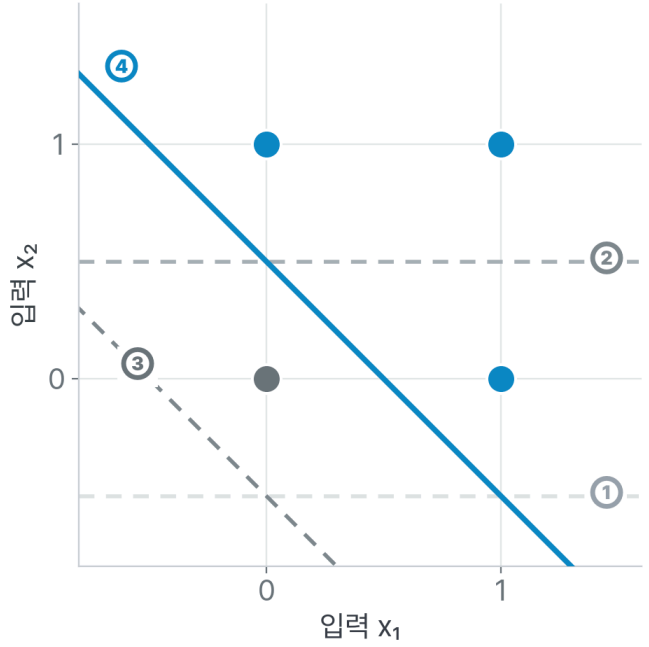
규칙은 다음과 같다.

$$w \leftarrow w + \eta\,(y - \hat{y})\,x, \qquad b \leftarrow b + \eta\,(y - \hat{y})$$

기호: η: 학습률(여기서 1) · y: 정답 · $\hat{y}$: 퍼셉트론의 출력

맞힌 점은 $y - \hat{y} = 0$ 이라 바뀌지 않는다. 1 을 0 으로 틀리면 $+x$, 0 을 1 로 틀리면 $-x$ 만큼 고친다. 점은 (0,0) → (0,1) → (1,0) → (1,1) 순서로 대입하고, 네 점을 한 번씩 모두 대입하는 것을 **에폭**(epoch)이라 한다.

고친 차례	에폭	틀린 점	z	고친 뒤 w	고친 뒤 b
①	1	(0,1)	−0.5	(0, 1)	0.5
②	2	(0,0)	0.5	(0, 1)	−0.5
③	2	(1,0)	−0.5	(1, 1)	0.5
④	3	(0,0)	0.5	**(1, 1)**	**−0.5**



OR 를 $w = (0, 0), b = -0.5$ 에서 시작했다. 네 번 고친 뒤 4에폭째에는 틀린 점이 없다. 선 번호는 표의 「고친 차례」와 같다

XOR 은 어떤 직선으로도 나눌 수 없어, 층이 하나인 퍼셉트론으로는 풀 수 없다

XOR: 두 입력이 다를 때만 1. (0,0)→0 · (0,1)→1 · (1,0)→1 · (1,1)→0

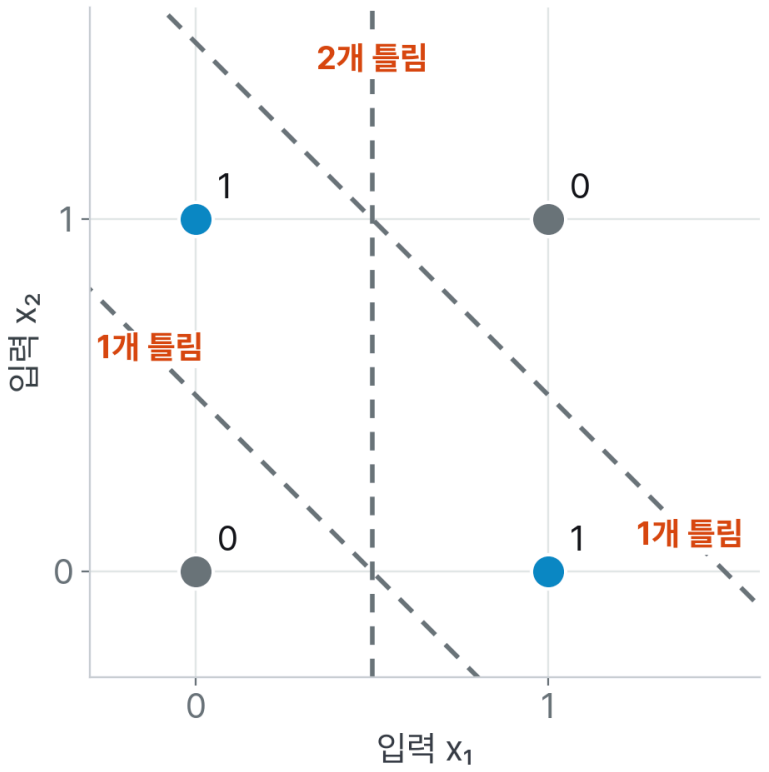
점마다 $z = w_1 x_1 + w_2 x_2 + b$ 를 적고, 출력이 같아야 하는 두 점의 부등식을 더한다.

출력이 1 이어야 하는 두 점: (0,1) 은 $w_2 + b > 0$, (1,0) 은 $w_1 + b > 0$. 더하면 $w_1 + w_2 + 2b > 0$

출력이 0 이어야 하는 두 점: (0,0) 은 $b \leq 0$, (1,1) 은 $w_1 + w_2 + b \leq 0$. 더하면 $w_1 + w_2 + 2b \leq 0$

같은 값 $w_1 + w_2 + 2b$ 가 0 보다 크면서 0 이하일 수는 없다

1969년 민스키·페퍼트의 『Perceptrons』가 층이 하나인 퍼셉트론의 이 한계를 정리했다. 층을 늘리면 XOR 을 풀 수 있다는 것은 알려져 있었지만, 늘린 층의 가중치를 학습하는 방법은 아직 없었다.



직선마다 적은 수: 직선의 어느 쪽을 1 로 정해도 남는 틀린 점의 최소 개수

정리: 퍼셉트론은 직선 하나로 나누는 모델이라 XOR 을 풀 수 없다

1 계산

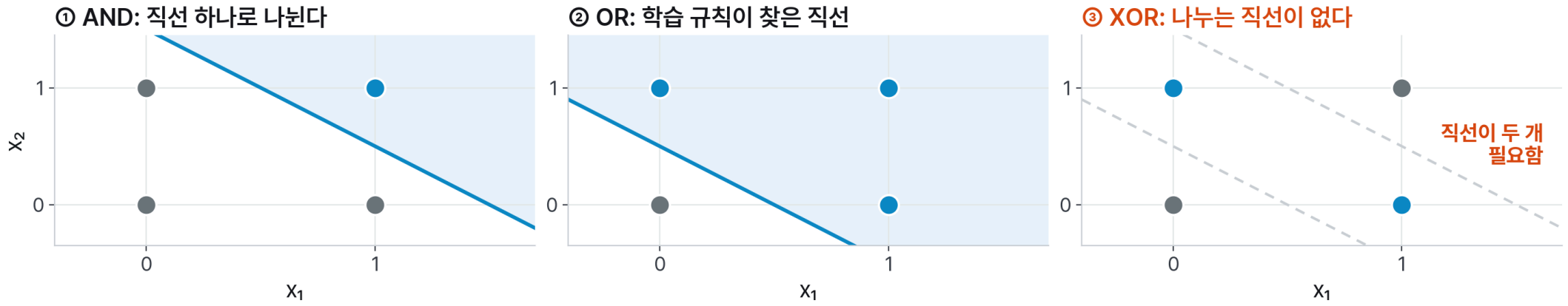
가중합 $z = w_1 x_1 + w_2 x_2 + b$ → 계단 함수 → 0 또는 1. 가중치·편향이 결정 경계 직선을 정한다

2 학습

틀린 점에서만 $\eta(y - \hat{y})x$ 만큼 고친다. AND·OR 는 이렇게 풀린다

3 한계

XOR 은 직선 하나로 나뉘지 않는다. $w_1 + w_2 + 2b$ 가 0 보다 크면서 0 이하일 수 없다



그림의 ①~③은 위 카드 1~3 에 해당한다. 파란 점: 출력 1, 회색 점: 출력 0

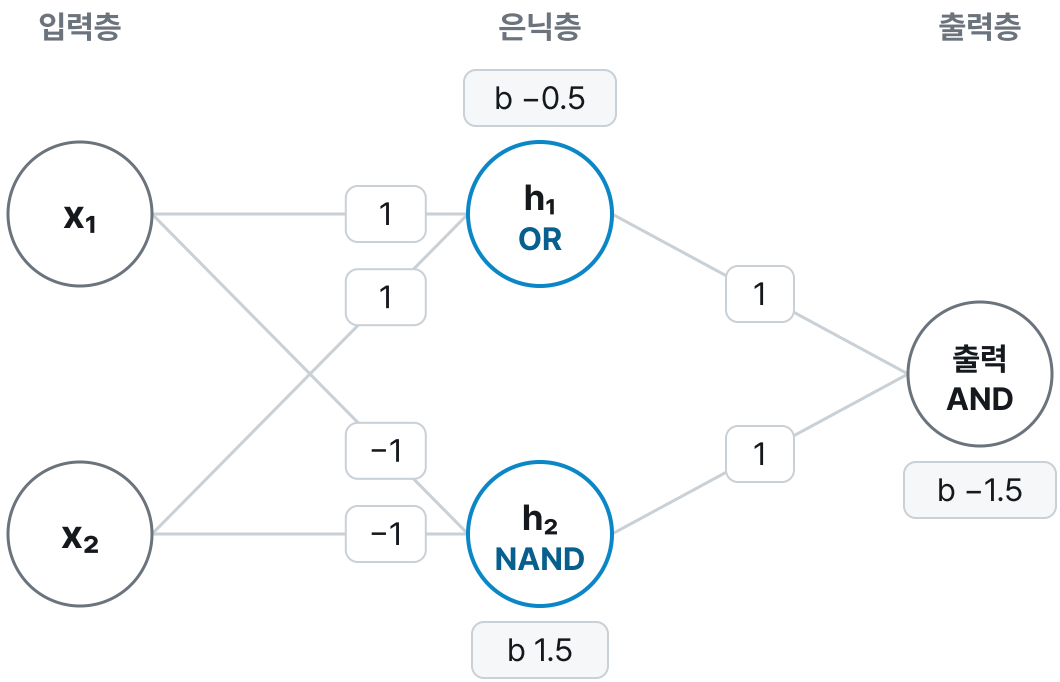
3

다층 퍼셉트론과 역전파

층을 하나 더하면 XOR 이 풀리는데, 그 층의 가중치는 어떻게 정하나?

은닉층이 하는 일, 1986년 역전파, 깊은 신경망의 기울기 소실과 2006년의 해법

은닉 노드 둘(OR·NAND)에 출력 노드 하나(AND)를 이으면 XOR 이 풀린다

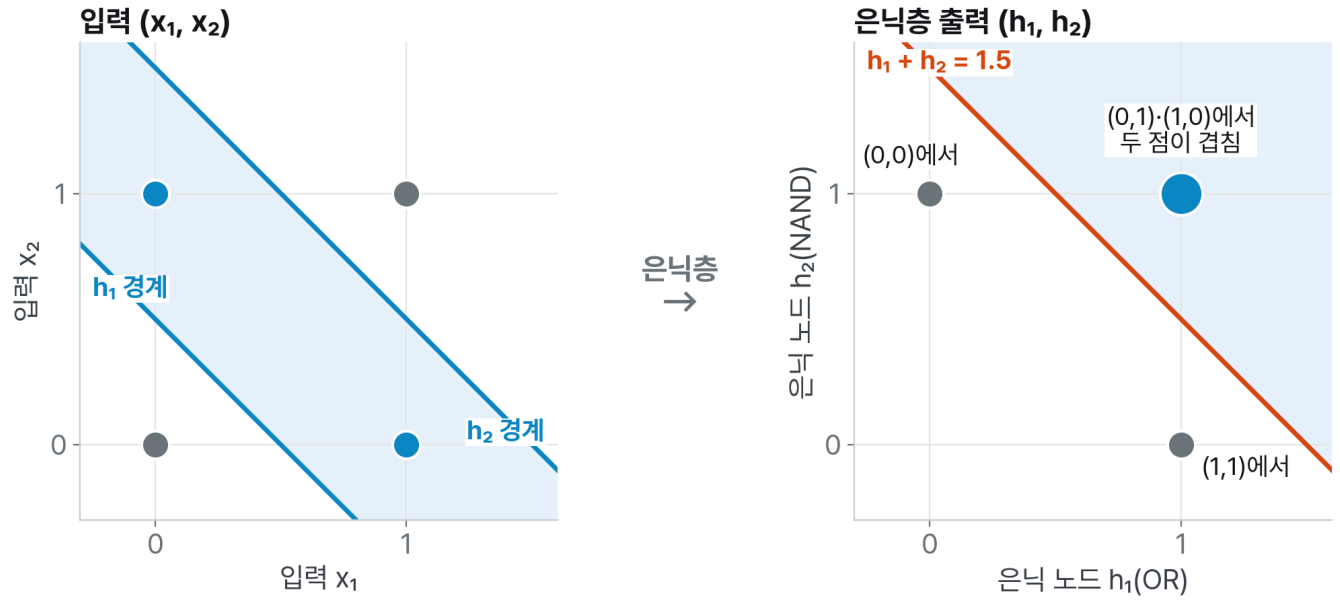


선 위의 수는 가중치, 회색 상자는 편향. NAND 는 AND 의 반대(둘 다 1 일 때만 0)

x_1	x_2	h_1 (OR)	h_2 (NAND)	출력(XOR)
0	0	0	1	0
0	1	1	1	**1**
1	0	1	1	**1**
1	1	1	0	0

- **은닉층**(hidden layer): 입력층과 출력층 사이의 층. 여기서는 노드 두 개
- 층이 여럿인 퍼셉트론을 **다층 퍼셉트론**(MLP, multi-layer perceptron)이라 부른다

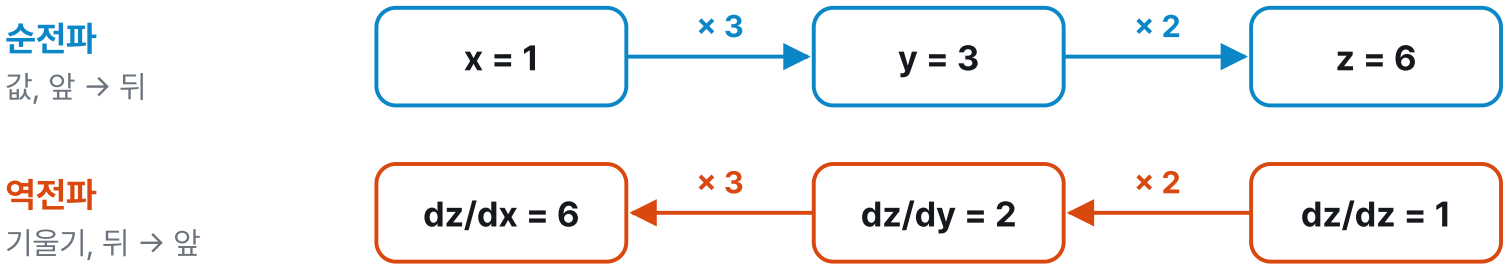
은닉층은 네 점의 좌표를 (h_1, h_2)로 바꿔, 직선 하나로 나뉘게 만든다



- 입력 평면에서는 XOR 값이 1 인 두 점이 두 경계 사이의 하늘색 영역에 있다:
  h_1 경계 $x_1 + x_2 = 0.5$,
  h_2 경계 $x_1 + x_2 = 1.5$
- 은닉층을 지나면 (0,0)→(0,1), (0,1)과 (1,0)→(1,1), (1,1)→(1,0). **XOR 값이 1 인 두 점이 같은 좌표 (1,1)이 된다**
- 바뀐 좌표에서는 직선 $h_1 + h_2 = 1.5$ 하나로 나뉜다. 출력 노드(AND)가 계산하는 경계가 이 직선이다

역전파는 손실에서 출발해, 단계마다의 비율을 곱해 앞쪽 가중치의 기울기를 구한다

- 1986년 러멜하트(David Rumelhart)·힌턴(Geoffrey Hinton)·윌리엄스(Ronald Williams)의 Nature 논문이 다층 신경망을 **역전파**(backpropagation)로 학습하는 법을 널리 알렸다
- **기울기**(gradient): 가중치를 조금 바꿀 때 손실(출력이 정답에서 빗나간 정도를 수 하나로 모은 값)이 바뀌는 비율. 이 비율을 알면 손실이 줄어드는 쪽으로 고칠 수 있다
- **연쇄 법칙**(chain rule): 이어진 계산의 비율은 곱한다. $y = 3x$, $z = 2y$ 이면 $\frac{dz}{dx} = 2 \times 3 = 6$. x 를 1 에서 1.1 로 늘리면 z 는 6 에서 **6.6** 이 된다

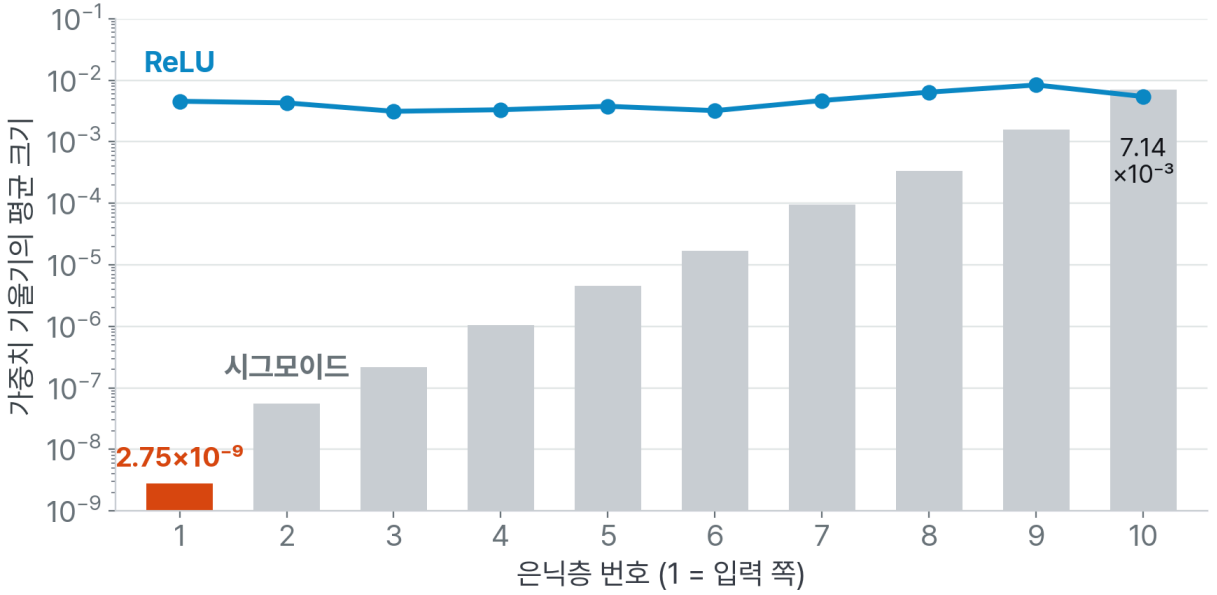


값은 앞에서 뒤로 계산하고(순전파, forward propagation), 기울기는 뒤에서 앞으로 곱해 온다(역전파). 「4. 순전파·역전파를 직접 계산하기」의 예에서는 가중치 w_5 → 출력 노드 o_1 의 가중합 → o_1 의 시그모이드 출력 → 손실 E 가 이 x → y → z 처럼 이어진다.

출처: Rumelhart, Hinton, Williams(1986), Learning representations by back-propagating errors, Nature 323, 533–536

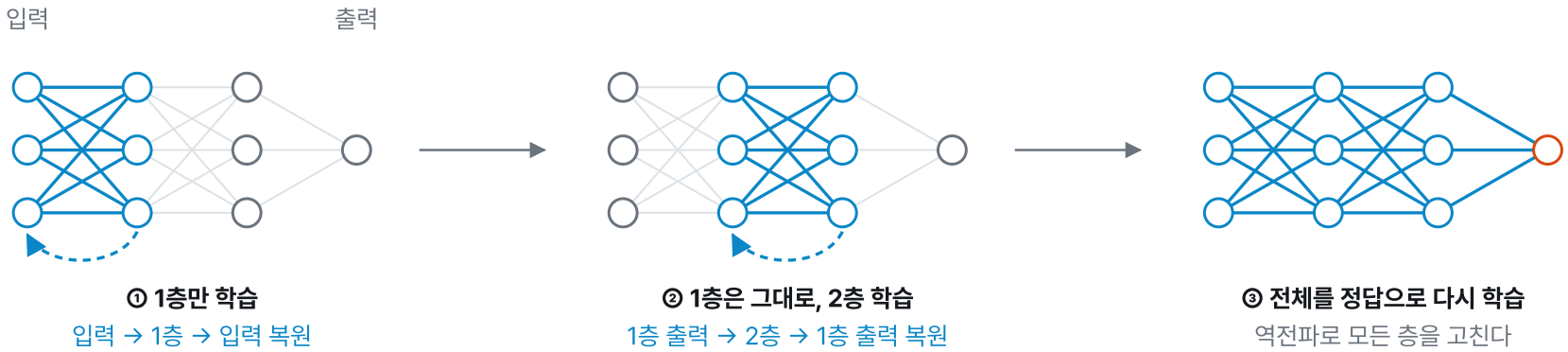
시그모이드를 층마다 지나면 기울기가 0.25 배 이하로 줄어, 앞쪽 층의 가중치가 거의 바뀌지 않는다

- **기울기 소실**(vanishing gradient): 역전파로 곱해 오는 수가 작아 앞쪽 층의 기울기가 0 에 가까워지는 현상. 1991년 호흐라이터(Sepp Hochreiter)의 학위 논문이 원인을 밝혔다
- 시그모이드 기울기는 가장 커도 0.25 라, 이 값을 층 10개만큼 곱하면 많아야 $0.25^{10} \approx$ 100만 분의 1
- 은닉층 10개 신경망에서 계산한 값: 시그모이드는 1층 기울기가 10층 기울기의 **약 260만 분의 1**. 가중합이 0 이 아니면 기울기가 0.25 보다 작아 이 상한보다도 작다
- ReLU(음수는 0, 양수는 그대로)에 맞춘 가중치 초기화(He 초기화)를 함께 쓰면 1층과 10층의 기울기가 비슷하다(1.2배)



은닉층 10개 × 노드 32개 + 출력 노드 1개, 무작위 입력·목표 256행, MSE 손실을 한 번 역전파한 뒤 층마다 가중치 기울기 절댓값의 평균(PyTorch, 시드 0). 세로축은 로그 스케일(눈금 하나가 10배). 가중치 초깃값은 시그모이드에 Xavier, ReLU 에 He 정규분포(편향 0). PyTorch 기본값으로 초깃값을 정하면 ReLU 도 1층 기울기가 10층 기울기의 약 400분의 1 이다

2006년 힌턴은 층마다 먼저 사전학습해 가중치 초깃값을 정하면 깊은 신경망도 학습된다는 것을 보였다



- 방법: 층을 하나씩 먼저 학습해 가중치 초깃값을 정한다. 이때 정답 대신, 층마다 그 층의 입력을 복원하도록 학습한다(입력이 곧 정답, 그림의 점선). 그다음 전체를 정답으로 다시 학습한다
- 이 방법을 **층별 사전학습**(layer-wise pretraining)이라 한다. 「8. 오늘날의 딥러닝」의 사전학습 모델(큰 자료로 미리 학습한 모델)과는 다른 방법이다
- 그 뒤 ReLU 와 GPU 학습이 퍼지면서 이 사전 단계 없이도 깊은 신경망을 학습하게 됐다(「8. 오늘날의 딥러닝」의 AlexNet, 2012)

출처: Hinton, Osindero, Teh(2006), A fast learning algorithm for deep belief nets, Neural Computation 18(7), 1527–1554 · 역전파로 전체를 다시 학습한 ③: Hinton, Salakhutdinov(2006), Reducing the dimensionality of data with neural networks, Science 313, 504–507

정리: 은닉층이 XOR 을 풀고, 은닉층의 가중치는 역전파로 학습한다

1 은닉층

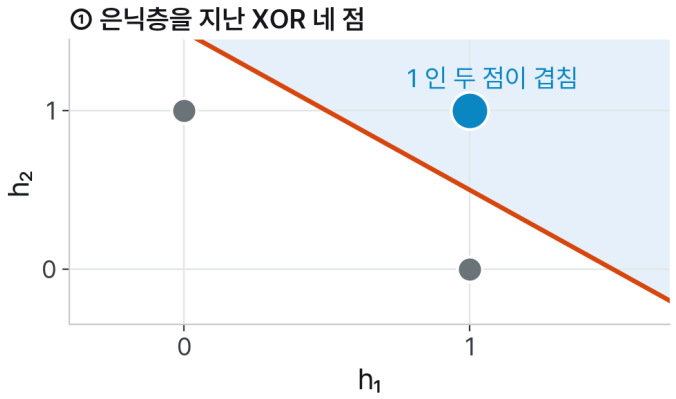
은닉 노드 둘(OR·NAND)이 네 점의 좌표를 바꾸면 출력 노드(AND)의 직선 하나로 XOR 이 나뉜다

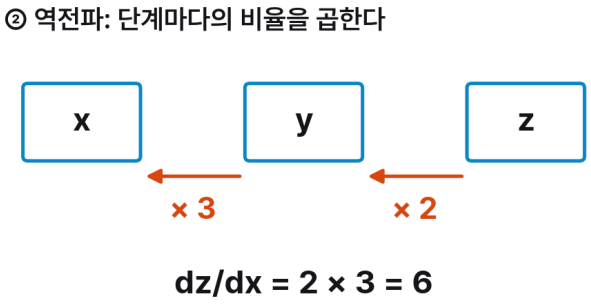
2 역전파

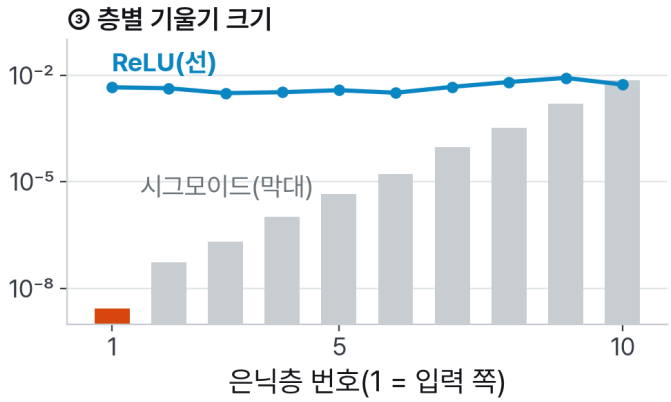
연쇄 법칙으로 손실에서 앞쪽 가중치까지 단계마다의 비율을 곱해 기울기를 구한다(1986)

3 기울기 소실

0.25 이하인 시그모이드 기울기가 층마다 곱해져 앞쪽 층의 기울기가 0 에 가까워진다. 층별 사전학습(2006)과 그 뒤의 ReLU 가 이 문제를 줄였다







4

순전파·역전파를 직접 계산하기

가중치 하나를 고치려면 어떤 수들을 곱해야 하나?

입력 2개·은닉 노드 2개·출력 노드 2개인 신경망에서 출력 0.7514·0.7729 를 구하고, w_5 와 w_1 을 학습률 0.5 로 한 번 고친다.

계산 예는 입력 2개·은닉 노드 2개·출력 노드 2개, 가중치 8개와 편향 2개를 둔 신경망이다

- 입력 $i_1 = 0.05, i_2 = 0.10$
  목표 $y_1 = 0.01, y_2 = 0.99$
- 편향: 은닉층 $b_1 = 0.35$(두 노드가 함께 씀), 출력층 $b_2 = 0.60$
- 활성화 함수: 시그모이드 · 학습률 0.5

가중치	연결	값	가중치	연결	값
w_1	$i_1 \to h_1$	0.15	w_5	$h_1 \to o_1$	0.40
w_2	$i_2 \to h_1$	0.20	w_6	$h_2 \to o_1$	0.45
w_3	$i_1 \to h_2$	0.25	w_7	$h_1 \to o_2$	0.50
w_4	$i_2 \to h_2$	0.30	w_8	$h_2 \to o_2$	0.55

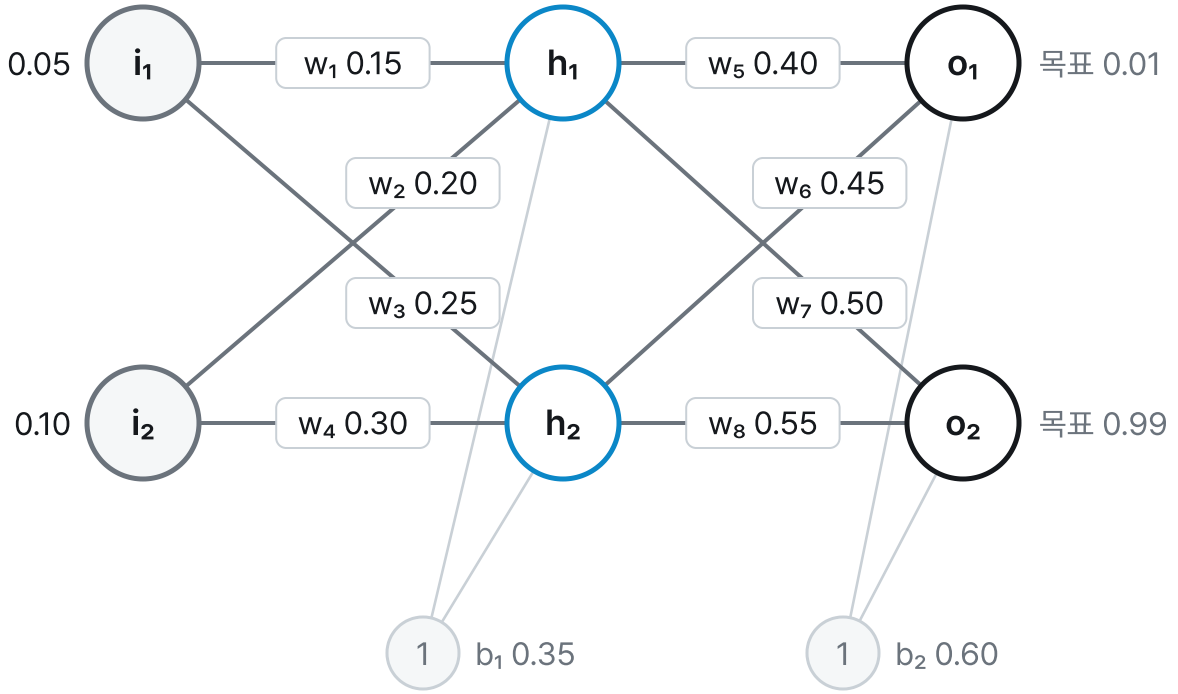


그림: 선 위 상자는 그 선의 가중치. 출처: Matt Mazur, 「A Step by Step Backpropagation Example」(2015), mattmazur.com

시그모이드는 어떤 수든 0 과 1 사이로 바꾸고, 이 예의 가중합은 0.38~1.22 사이다

시그모이드(sigmoid)는 다음과 같다.

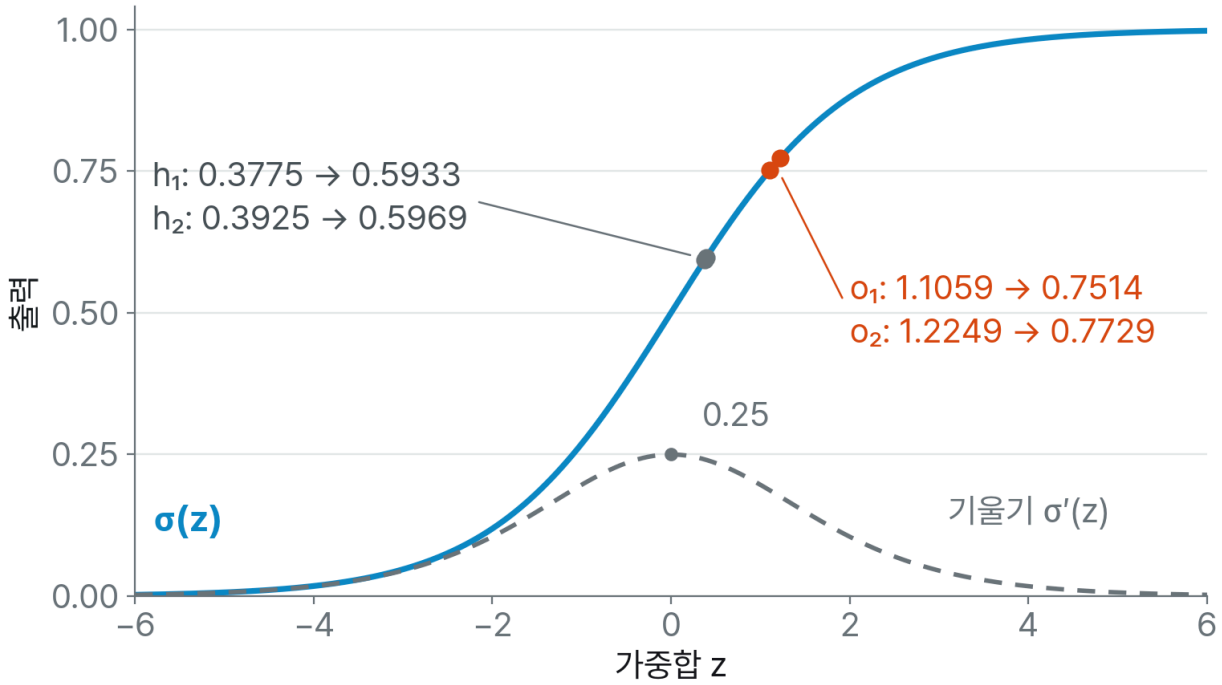
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

z: 노드의 가중합 · e: 자연로그의 밑(약 2.718)

- $\sigma(0) = 0.5 \cdot \sigma(2) = 0.881 \cdot \sigma(-2) = 0.119$
- 기울기는 시그모이드 출력만으로 계산한다. $z = 0$ 에서 $0.5 \times 0.5 = 0.25$ 로 가장 크다.

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

- 이 예의 가중합: h_1 0.3775 · h_2 0.3925 · o_1 1.1059 · o_2 1.2249



점: 이 예의 네 가중합(h_1·h_2 회색, o_1·o_2 주황)과 시그모이드를 거친 값

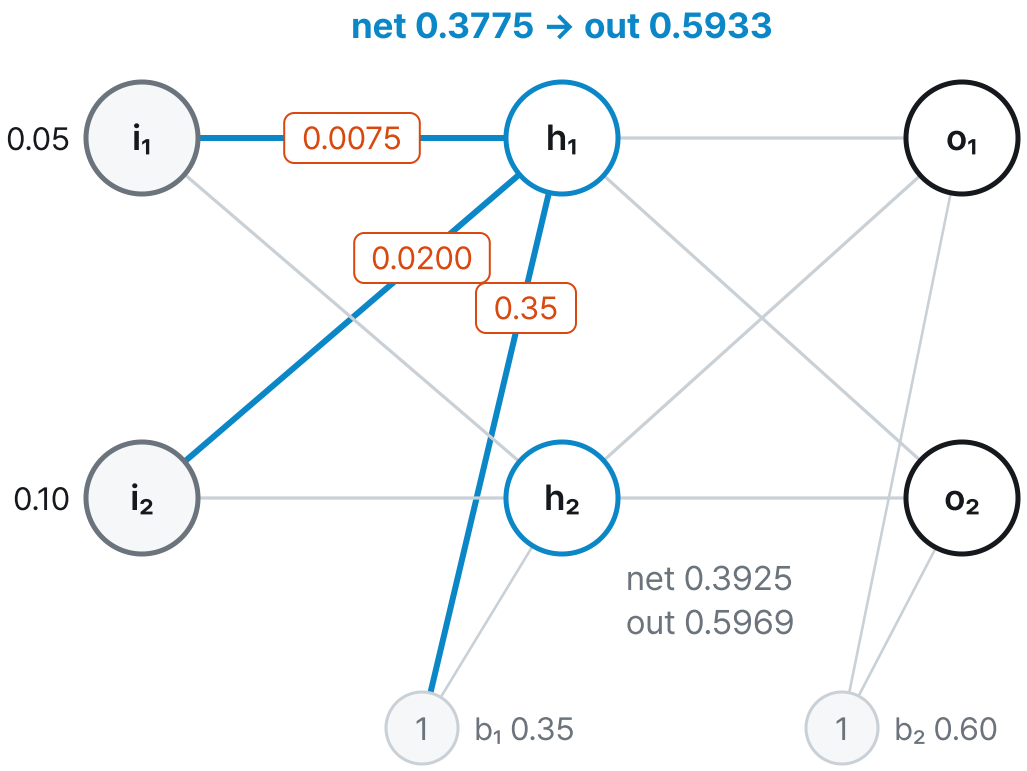
은닉 노드 h_1 의 가중합은 0.3775 이고, 시그모이드를 거친 출력은 0.5933 이다

h_1 의 가중합은 다음과 같다.

$$net_{h_1} = w_1 i_1 + w_2 i_2 + b_1$$

net: 노드의 가중합 · out: 시그모이드를 거친 노드의 출력

- 대입: $0.15 \times 0.05 + 0.20 \times 0.10 + 0.35$
  $= 0.0075 + 0.0200 + 0.35 = 0.3775$
  $\rightarrow out_{h_1} = \sigma(0.3775) =$ **0.5933**
- h_2 도 같은 방법으로: $0.0125 + 0.0300 + 0.35 = 0.3925$
  $\rightarrow out_{h_2} = 0.5969$



주황 상자: 가중치 × 입력(0.15 × 0.05 = 0.0075, 0.20 × 0.10 = 0.0200)과 편향 0.35

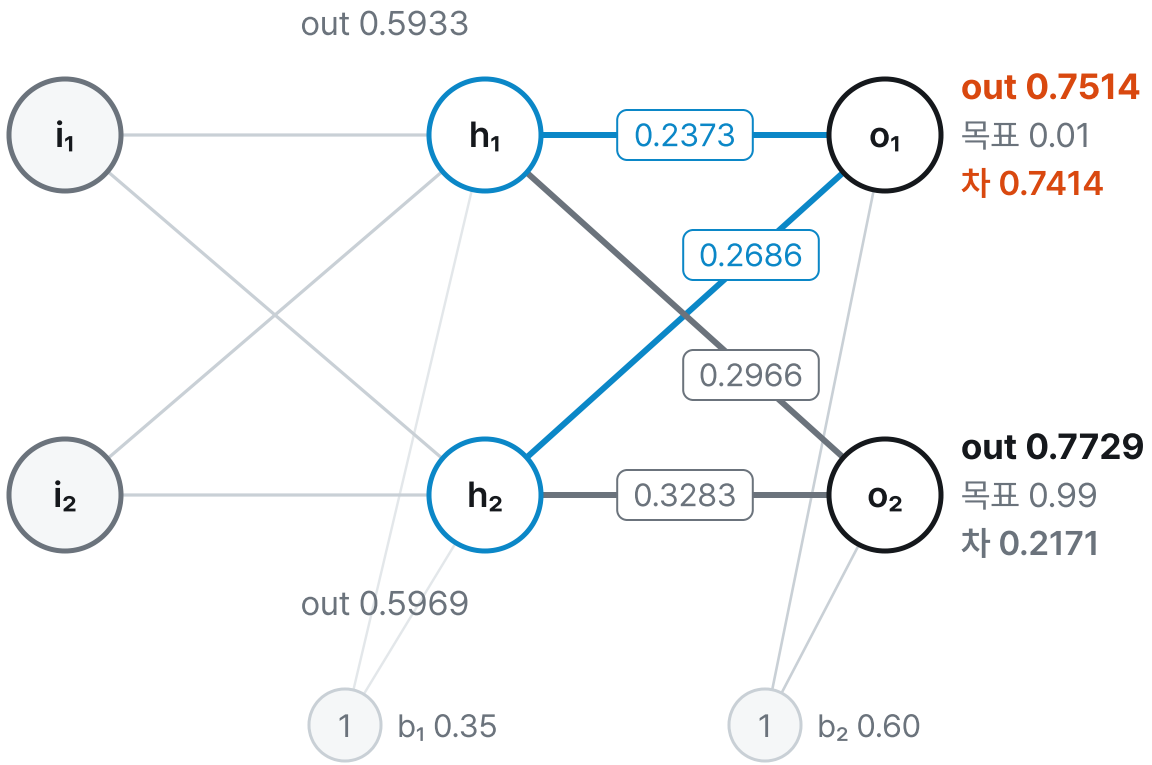
출력 노드 o_1 의 출력은 0.7514, o_2 는 0.7729 로 목표 0.01·0.99 와 차이가 난다

o_1 의 가중합은 다음과 같다.

$$net_{o_1} = w_5\, out_{h_1} + w_6\, out_{h_2} + b_2$$

$$\begin{aligned} net_{o_1} &= 0.40 \times 0.5933 + 0.45 \times 0.5969 + 0.60 \\ &= 0.2373 + 0.2686 + 0.60 = 1.1059 \end{aligned}$$

- $out_{o_1} = \sigma(1.1059) = \mathbf{0.7514}$
- o_2 도 같은 방법으로: $0.2966 + 0.3283 + 0.60 = 1.2249$
  $\rightarrow out_{o_2} = 0.7729$
- o_1 은 목표보다 0.7414 크고, o_2 는 목표보다 0.2171 작다



파랑 상자: o_1 로 들어가는 두 곱, 회색 상자: o_2 로 들어가는 두 곱

손실은 0.2984 이고, 그 가운데 0.2748 이 o_1 에서 나온다

이 예의 **손실** E 는 출력마다 제곱 오차의 절반을 더한 값이다. 출력이 정답에서 빗나간 정도를 수 하나로 모은 것이다(「6. 손실 함수」).

$$E = \frac{1}{2}(y_1 - out_{o_1})^2 + \frac{1}{2}(y_2 - out_{o_2})^2$$

y_1, y_2: 목표 · out_{o_1}, out_{o_2}: 출력

- 대입: $\frac{1}{2}(0.01 - 0.7514)^2 + \frac{1}{2}(0.99 - 0.7729)^2$
  $= \frac{1}{2}(0.5496) + \frac{1}{2}(0.0471)$
  $= 0.2748 + 0.0236 = \mathbf{0.2984}$
- ½ 은 미분할 때 지수 2 와 약분되도록 곱해 두는 관례다. 출력이 2개라 이 값은 제곱 오차의 평균(MSE)과도 같다



w_5 의 기울기는 w_5 → net_o_1 → out_o_1 → E 로 이어지는 세 비율의 곱이다

E 의 식에는 w_5 가 바로 나오지 않는다. w_5 는 net_o_1, out_o_1 을 거쳐 E 를 바꾼다. 그래서 연쇄 법칙으로 세 비율을 곱한다.

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial out_{o_1}} \cdot \frac{\partial out_{o_1}}{\partial net_{o_1}} \cdot \frac{\partial net_{o_1}}{\partial w_5}$$

∂(편미분): 다른 가중치는 그대로 두고 이 하나만 조금 바꿀 때의 비율. 읽는 법은 「3. 다층 퍼셉트론과 역전파」의 d 와 같다

순전파: 값이 왼쪽에서 오른쪽으로 · 역전파: 비율이 오른쪽에서 왼쪽으로



① $\frac{1}{2}(y_1 - out)^2$ 을 out 으로 미분: $\frac{1}{2} \cdot 2(y_1 - out) \cdot (-1) = -(y_1 - out)$, ½ 은 지수 2 와 약분된다(앞 장). $-(0.01 - 0.7514) = 0.7414$
검산: out 0.7514 → 0.7524(+0.001)면 o_1 항 0.27484 → 0.27558, 비율 0.00074 ÷ 0.001 ≈ 0.74

② $out_{o_1}(1 - out_{o_1})$ $= 0.7514 \times 0.2486 = 0.1868$

③ w_5 에 곱해지는 입력 $out_{h_1} = 0.5933$

세 비율을 곱하면 0.0822 이고, 학습률 0.5 로 w_5 를 갱신하면 0.40 에서 0.3589 가 된다

0.7414 × 0.1868 = δ_o₁ 0.1385 × 0.5933 = 기울기 0.0822 → × 0.5 = 0.0411 → w_5: 0.40 → 0.3589

기울기 0.0822 는 양수이므로 w_5 를 줄이면 E 가 줄어든다. 갱신 식은 다음과 같다.

$$w_5^{+} = w_5 - \eta \frac{\partial E}{\partial w_5}$$



η: 학습률(0.5) · w_5^{+}: 고친 뒤의 w_5

대입: $0.40 - 0.5 \times 0.0822 = 0.40 - 0.0411 = \mathbf{0.3589}$

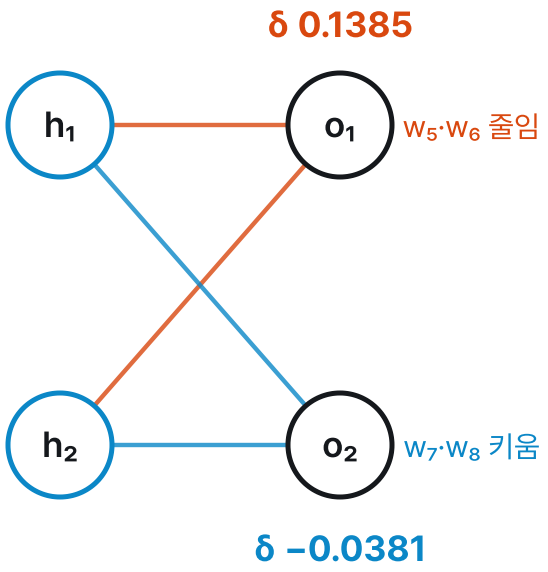
> 앞 두 비율의 곱 $0.7414 \times 0.1868 = 0.1385$ 를 o_1 의 오차 신호 δ_{o_1} 이라 부른다. o_1 로 들어가는 가중치의 기울기는 모두 $\delta_{o_1} \times$(들어오는 값)이다.

출력층 가중치 넷은 모두 「오차 신호 × 들어오는 값」으로 기울기를 구해 한 번에 고친다

- o_2 의 오차 신호: $\delta_{o_2} = -(0.99 - 0.7729) \times 0.7729 \times (1 - 0.7729) = -0.2171 \times 0.1755 = -0.0381$
- δ 가 음수이므로 o_2 로 들어가는 w_7·w_8 은 커진다. 그러면 o_2 의 출력이 커져 목표 0.99 에 가까워진다

가중치	연결	δ	들어오는 값	기울기	지금 → 새 값
w_5	$h_1 \to o_1$	0.1385	0.5933	0.0822	0.40 → **0.3589**
w_6	$h_2 \to o_1$	0.1385	0.5969	0.0827	0.45 → **0.4087**
w_7	$h_1 \to o_2$	−0.0381	0.5933	−0.0226	0.50 → **0.5113**
w_8	$h_2 \to o_2$	−0.0381	0.5969	−0.0227	0.55 → **0.5614**

기울기 = δ × 들어오는 값 · 새 값 = 지금 값 − 0.5 × 기울기 · 주황: 줄어든 값, 파랑: 늘어난 값

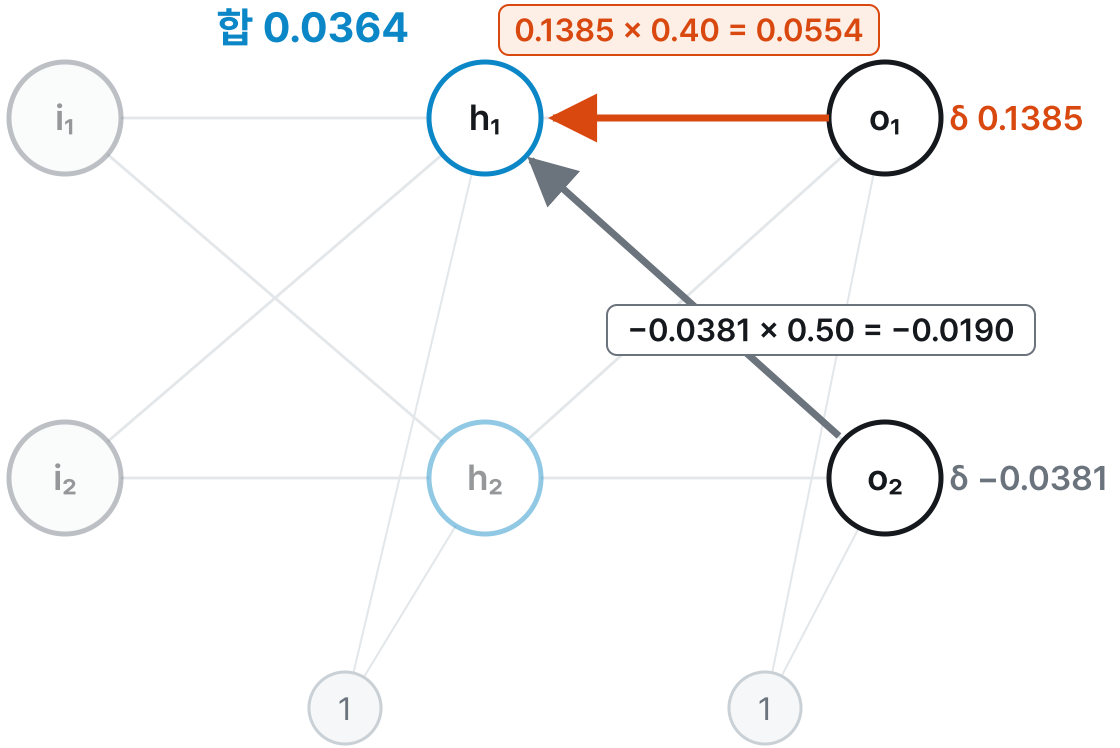


h_1 의 출력은 o_1 과 o_2 둘의 입력이므로, 두 경로의 항을 더하면 0.0364 다

h_1 의 출력이 E 를 바꾸는 비율은 o_1 을 거치는 항과 o_2 를 거치는 항의 합이다.

$$\frac{\partial E}{\partial out_{h_1}} = \delta_{o_1} w_5 + \delta_{o_2} w_7$$

- 대입: $0.1385 \times 0.40 + (-0.0381) \times 0.50 = 0.0554 - 0.0190 = \mathbf{0.0364}$
- w_5·w_7 은 고치기 전 값(0.40·0.50)을 쓴다. 기울기를 모두 구한 뒤 한꺼번에 고치기 때문이다
- h_2 도 같은 방법으로: $0.0623 - 0.0210 = 0.0414$

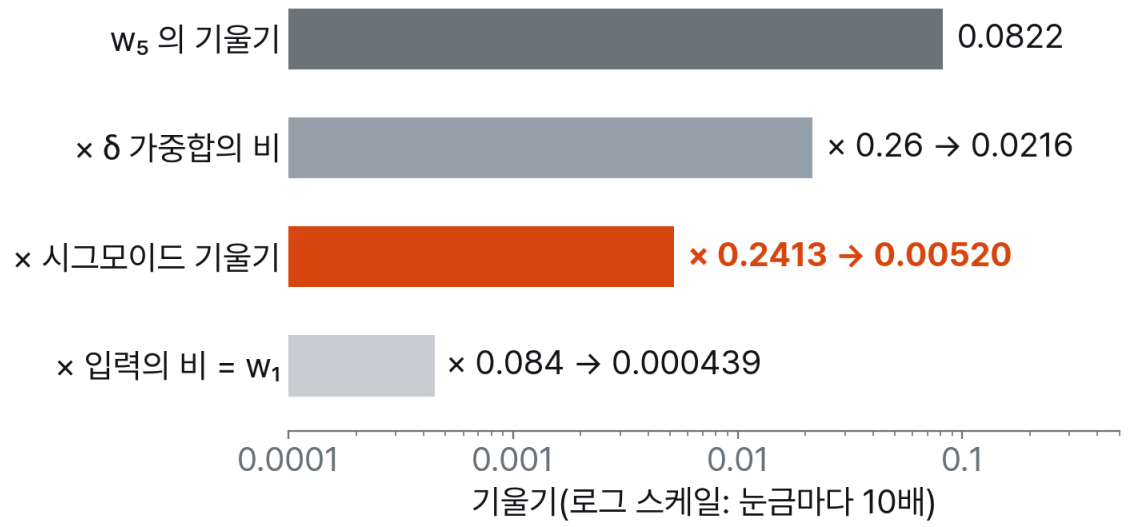


화살표: 역전파 방향(출력 → 은닉). 상자: 오차 신호 δ × 고치기 전 가중치

w_1 의 기울기는 0.000439 로 w_5 의 약 187분의 1 이라, w_1 은 0.15 에서 0.14978 로 조금만 바뀐다

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out_{h_1}} \cdot \frac{\partial out_{h_1}}{\partial net_{h_1}} \cdot \frac{\partial net_{h_1}}{\partial w_1} = 0.0364 \times \underbrace{0.5933 \times (1 - 0.5933)}_{0.2413} \times \underbrace{0.05}_{i_1} = 0.000439$$

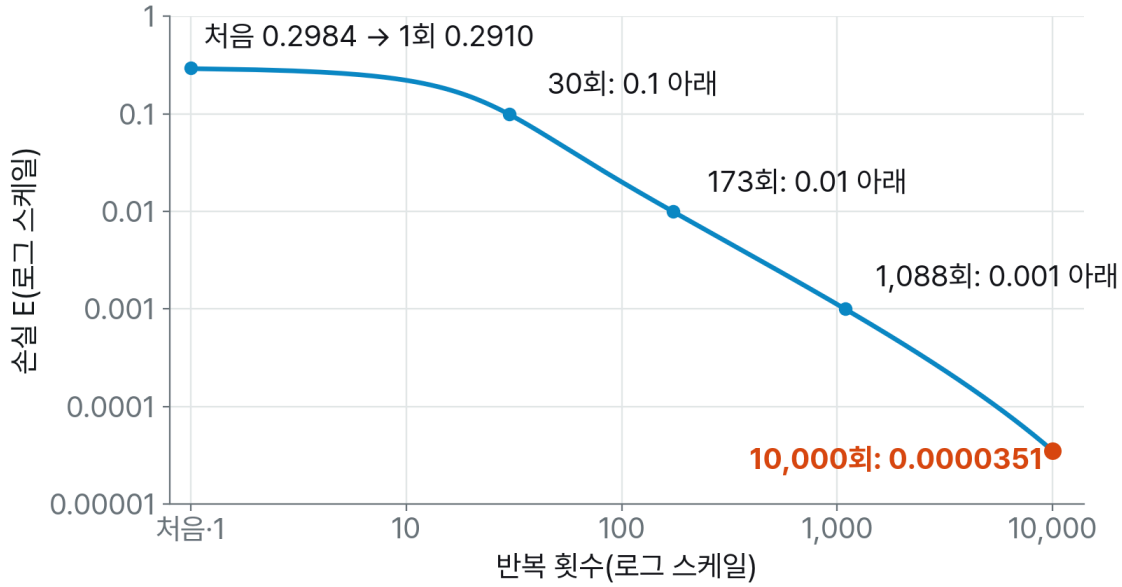
- 갱신: $0.15 - 0.5 \times 0.000439 = \mathbf{0.14978}$
- w_5 기울기 0.0822 에서 w_1 기울기까지 곱해지는 수는 셋이다: δ 가중합($\delta_{o_1} w_5 + \delta_{o_2} w_7$)과 δ_{o_1} 의 비 0.0364 ÷ 0.1385 ≈ 0.26배, **시그모이드 기울기 0.2413배**, 입력의 비 0.05 ÷ 0.5933 ≈ 0.084배. 셋을 곱하면 187분의 1
- 층이 하나 늘 때마다 첫째·둘째 수가 한 번씩 더 곱해진다. 시그모이드 기울기는 늘 0.25 이하라, 층이 많을수록 앞쪽 층의 기울기가 0 에 가까워진다(기울기 소실). 셋째 수(입력의 비)는 맨 앞 층에서 한 번만 곱해진다



w_5 기울기(0.7414 × 0.1868 × 0.5933)에 세 수를 차례로 곱한 값

가중치 8개를 모두 고치면 손실이 0.2984 에서 0.2910 으로 줄고, 10,000번 되풀이하면 손실은 0.0000351 이 된다

- 은닉층 나머지: w_2 0.20 → 0.19956 · w_3 0.25 → 0.24975 · w_4 0.30 → 0.29950
- 새 가중치로 다시 순전파: o_1 0.7514 → 0.7421, o_2 0.7729 → 0.7753, E 0.2984 → **0.2910**
- 갱신을 되풀이하면 손실이 30번 만에 0.1, 173번 만에 0.01, 1,088번 만에 0.001 아래로 내려간다. 10,000번 뒤 출력은 0.0159·0.9841(목표 0.01·0.99)



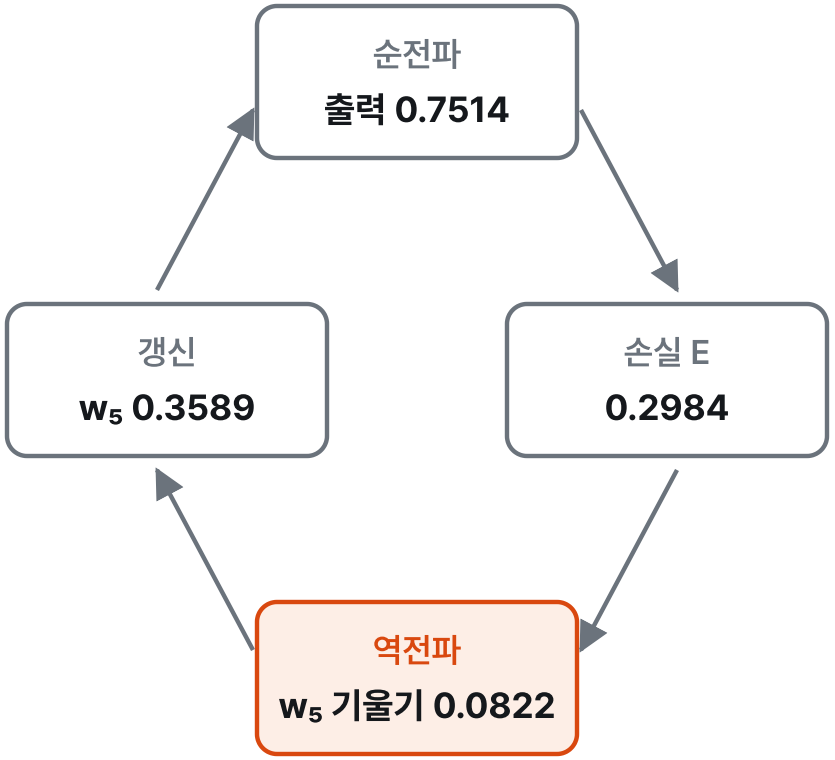
편향 0.35·0.60 은 Mazur 의 원래 예처럼 고정, 학습률 0.5

학습 한 번의 구조는 이렇다.

순전파(값) → **손실 E** → **역전파(기울기)** → **갱신**

정리: 가중치 하나의 기울기는 손실에서 그 가중치까지 이어진 경로마다 비율을 곱해 더한 값이다

- **순전파**: 층마다 가중합 → 시그모이드. 이 예의 출력 0.7514·0.7729, 손실 0.2984
- **역전파**: 출력층 기울기 = δ × 들어오는 값(w_5 0.0822). 은닉층 기울기는 δ 가중합에 시그모이드 기울기와 입력을 곱한다(w_1 0.000439)
- **갱신**: $w^{+} = w - 0.5 \times$기울기. 한 번에 0.2984 → 0.2910, 10,000번에 0.0000351



5

신경망의 부품

신경망 하나를 만들 때 무엇을 정해야 하나?

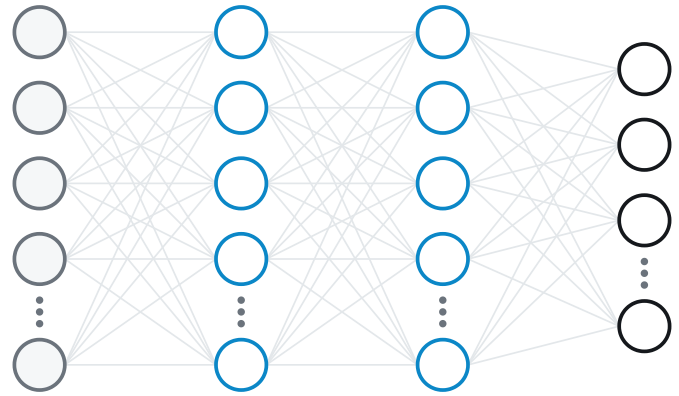
층마다 노드 수 · 학습 중에 계산되는 파라미터와 학습 전에 정하는 하이퍼파라미터 · 편향과 활성화 함수가 출력을 바꾸는 방식

입력 노드 수는 특성 수로, 출력 노드 수는 예측할 값의 개수로 정해진다

- 입력층: 특성 하나에 노드 하나. 입력이 지난 28일 승차면 노드 28개
- 은닉층: 층 수와 노드 수는 학습 전에 정한다(이 과정의 MLP: 64개씩 2층). 은닉 노드 하나의 값은 다음과 같다.

$$h_j = f\left(\sum_k w_{jk} x_k + b_j\right)$$

x_k: k번째 입력 · w_{jk}: 입력 k 에서 노드 j 로 가는 가중치 · b_j: 노드 j 의 편향 · f: 활성화 함수



이 과정의 MLP: 지난 28일 → 64 → 64 → 다음 7일

문제 유형	출력 노드	출력층 활성화	예
회귀, 값 하나	1	없음(그대로)	내일 승차
회귀, 값 여러 개	예측할 값 수	없음	다음 7일 승차 → **7**
이진 분류	1	시그모이드	고객 이탈 여부
다중 분류(K개 범주)	K	소프트맥스	문의 유형 3종 → 3

가중치·편향은 학습 중에 계산되는 파라미터이고(이 과정 MLP 6,471개), 층 수·학습률은 학습 전에 정하는 하이퍼파라미터다

층 하나의 파라미터 수 = 입력 수 × 출력 수(가중치) + 출력 수(편향). 예: 28→64 는 $28 \times 64 + 64 = 1{,}856$



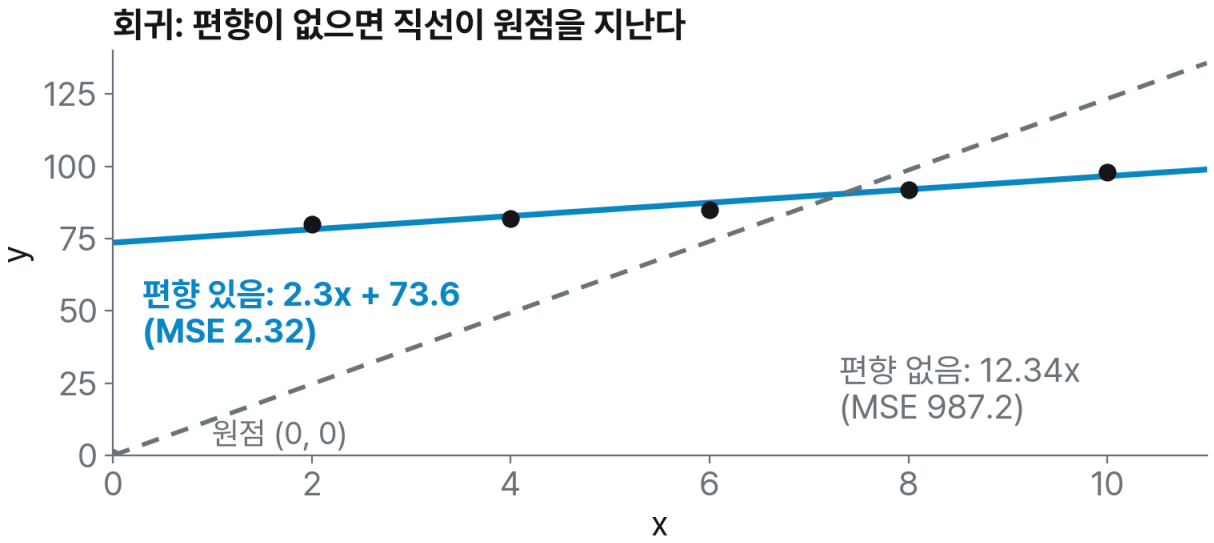
구분	파라미터(parameter)	하이퍼파라미터(hyperparameter)
정하는 때	학습 중(경사하강법이 계산)	학습 전(직접 지정)
예	가중치 · 편향	은닉층 수 · 노드 수 · 활성화 함수 · 손실 함수 · 학습률 · 배치 크기 · 에폭 수 · 조기 종료 기준
이 과정 MLP	**6,471개**	은닉 64 · 64, ReLU, 학습률 0.001, 배치 256
순전파·역전파 계산 예	가중치 8 + 편향 2 = 10개	은닉 노드 2, 시그모이드, 학습률 0.5

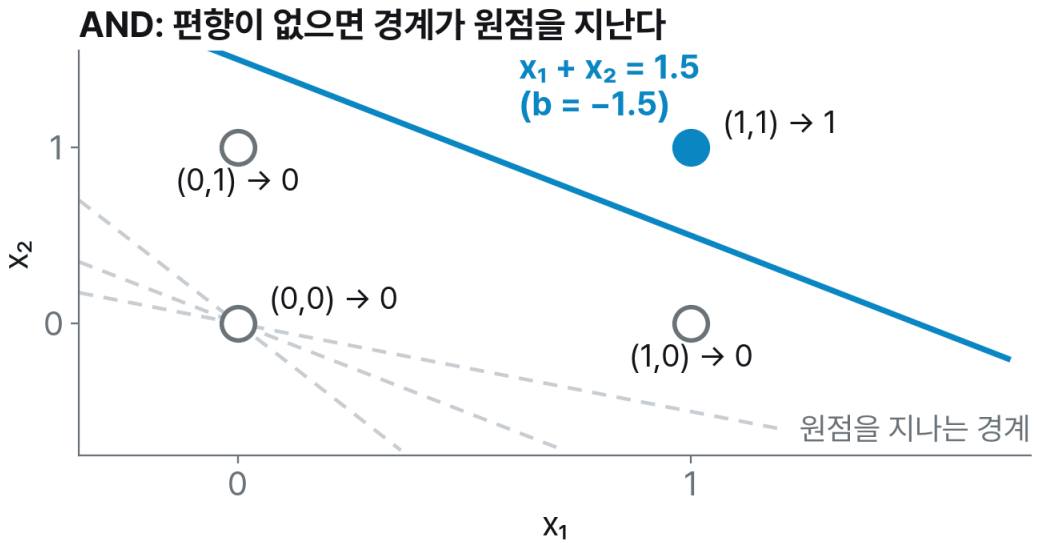
편향이 없으면 직선과 경계가 원점을 지나야 해서, 선형회귀는 크게 빗나가고(MSE 2.32 → 987.2) AND 는 풀 수 없다

편향(bias) b: 입력이 모두 0 일 때의 가중합 z 의 값. 회귀 직선을 위아래로 옮기고, 결정 경계를 평행 이동한다.

회귀 예(x = 2, 4, 6, 8, 10 · y = 80, 82, 85, 92, 98)
편향 있음 $\hat{y} = 2.3x + 73.6$, MSE **2.32**
편향 없음 $\hat{y} = 12.34x$, MSE **987.2**

AND 예: 편향이 0 이면 $z(1,0) = w_1 \le 0$, $z(0,1) = w_2 \le 0$ 이어야 하고, 그러면 $z(1,1) = w_1 + w_2 \le 0$ 이라 (1,1) 에서 1 을 낼 수 없다. $b = -1.5$ 이면 풀 수 있다

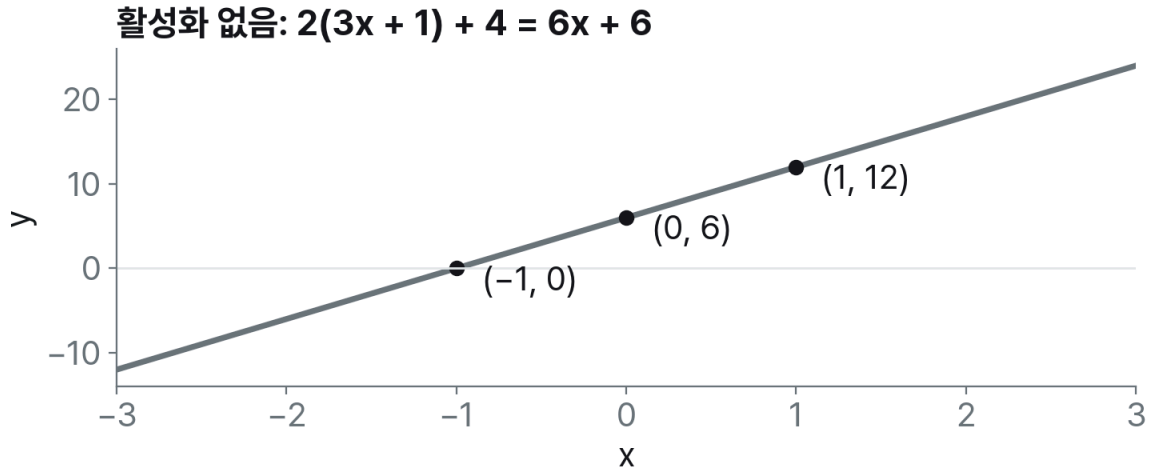


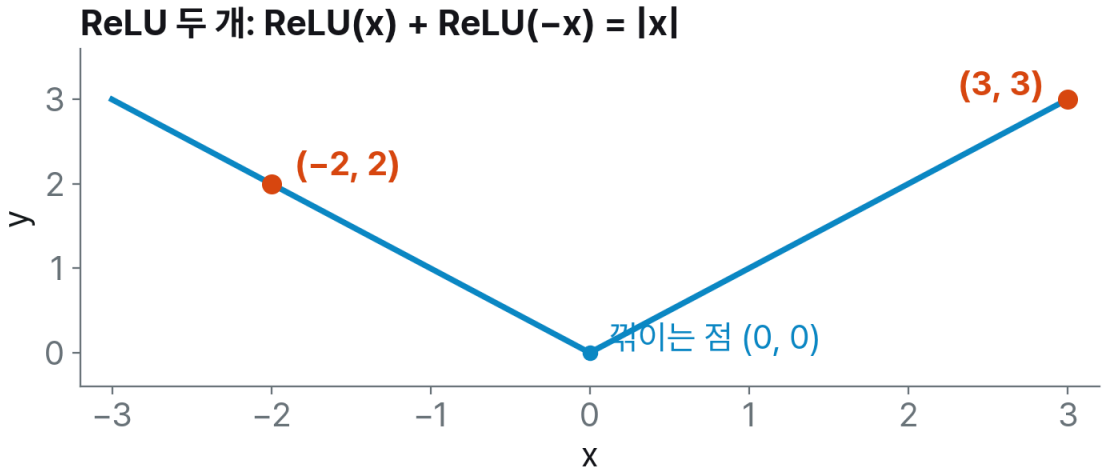


활성화 함수가 없으면 층을 몇 개 이어도 직선 하나와 같다

선형 층 둘: $h = 3x + 1$, $y = 2h + 4$
→ $y = 2(3x + 1) + 4 = 6x + 6$. 층이 둘이어도 식은 직선 하나다

비선형 활성화 함수를 적용하면 직선이 꺾인다: $y = \mathrm{ReLU}(x) + \mathrm{ReLU}(-x) = |x|$
$x = -2$ 이면 $0 + 2 = 2$, $x = 3$ 이면 $3 + 0 = 3$



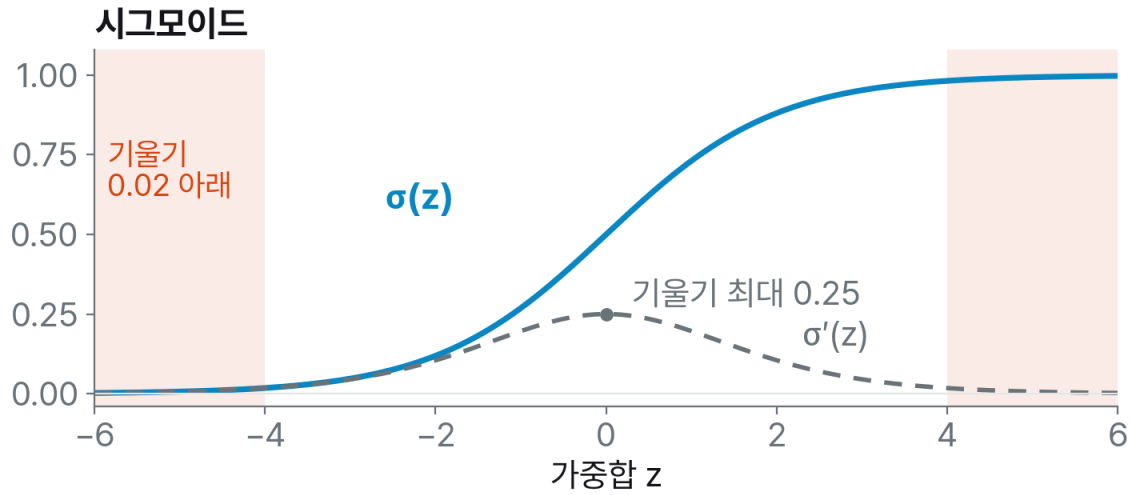


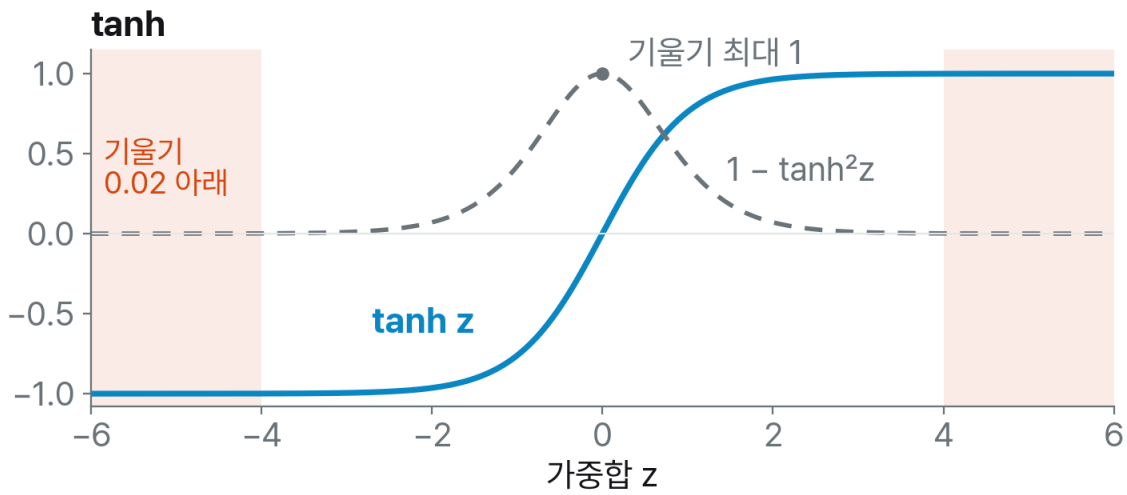
활성화 함수(activation function): 노드의 가중합에 적용하는 비선형 함수. 노드를 늘리면 꺾이는 점이 늘어 곡선 모양에도 가깝게 맞출 수 있다.

시그모이드와 tanh 는 입력이 크거나 작으면 기울기가 0 에 가까워진다

시그모이드 $\sigma(z) = 1/(1+e^{-z})$: 출력 0~1
기울기 $\sigma(1-\sigma)$ 는 $z=0$ 에서 최대 0.25, $z=\pm 4$ 에서 0.018

tanh(하이퍼볼릭 탄젠트) $\tanh z = \dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$: 출력 -1~1
기울기 $1-\tanh^2 z$ 는 $z=0$ 에서 최대 1, ± 2 에서 0.071, ± 4 에서 0.0013



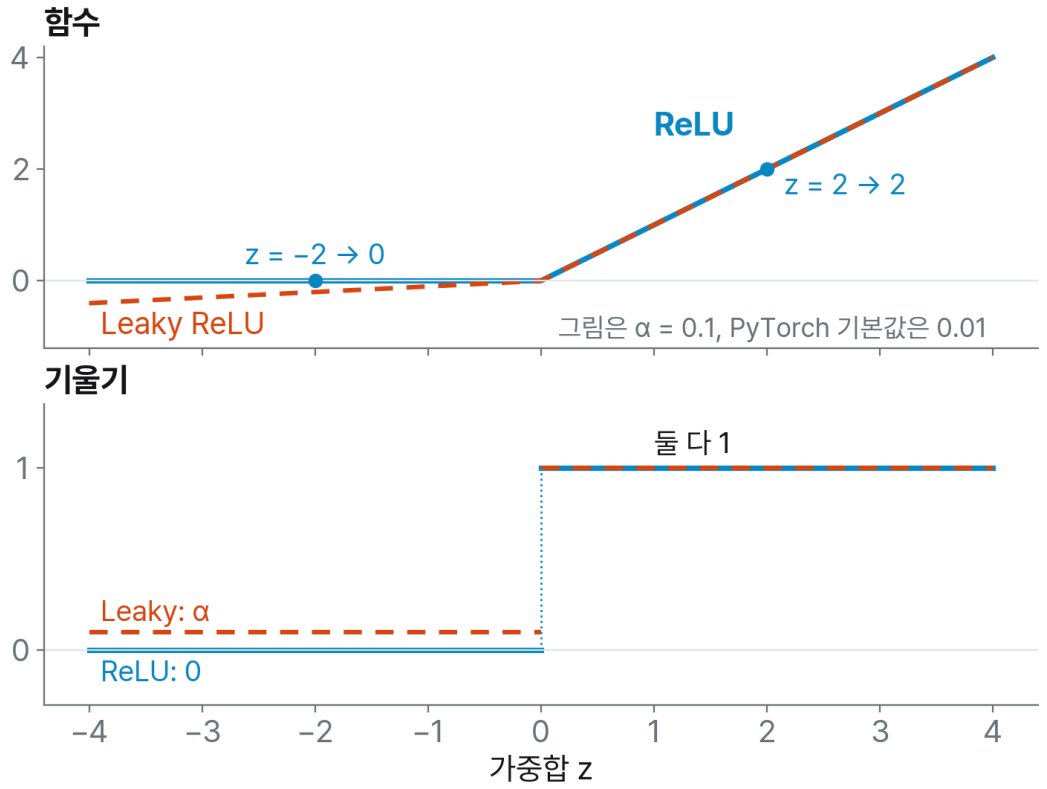


파랑 실선: 함수, 회색 점선: 기울기, 주황 영역: |z| > 4 구간

tanh 의 최대 기울기는 시그모이드의 4배지만, 양 끝이 평평해 층이 깊으면 기울기가 0 에 가까워지는 것은 같다.

ReLU 는 양수에서 기울기가 1 이라 층을 여러 개 거쳐도 활성화 함수 때문에 기울기가 줄지는 않고, Leaky ReLU 는 음수에서도 기울기가 0.01 이다

- ReLU(rectified linear unit) $\max(0, z)$: $z = -2 \rightarrow 0$, $z = 2 \rightarrow 2$. 기울기는 음수에서 0, 양수에서 1
- 죽은 ReLU(dying ReLU): 가중합이 모든 행에서 음수인 노드는 기울기가 늘 0 이라, 그 노드로 들어가는 가중치가 더 바뀌지 않는다
- Leaky ReLU $\max(\alpha z, z)$: PyTorch 기본 $\alpha = 0.01$, $z = -2 \rightarrow -0.02$. 음수에서도 기울기가 0.01 이라 가중치가 계속 바뀔 수 있다
- 이 과정의 MLP 는 은닉층에 ReLU 를 쓴다



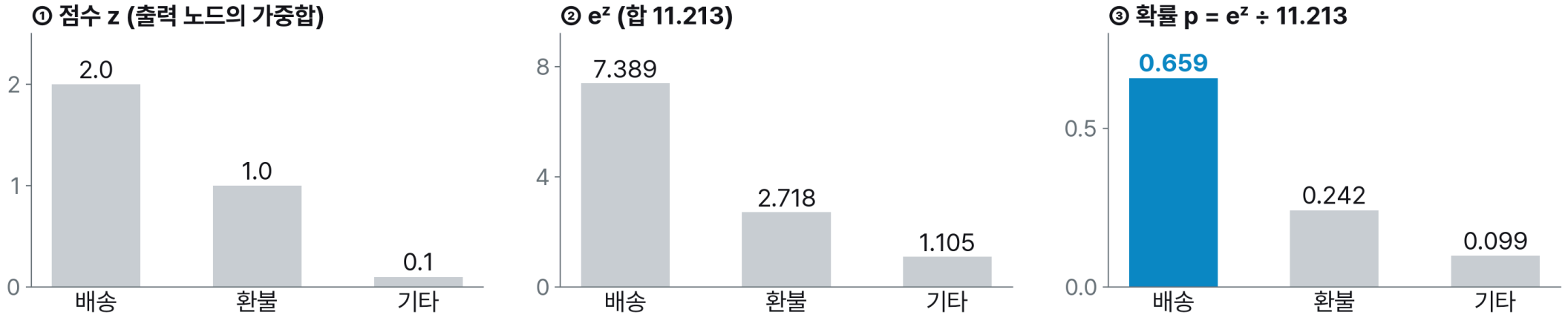
소프트맥스는 점수 여러 개를 합이 1 인 확률로 바꾼다

소프트맥스(softmax)는 다음과 같다.

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

z_k: 범주 k 의 점수, 곧 출력 노드 k 의 가중합(활성화 전 값) · p_k: 범주 k 의 확률

- 문의 유형 배송·환불·기타의 점수 2.0, 1.0, 0.1 → e^z = 7.389, 2.718, 1.105(합 11.213) → 확률 0.659, 0.242, 0.099
- 점수 순서는 그대로이고 합은 1 이다. 그래서 다중 분류의 출력층에 쓴다
- 점수 여러 개를 받는 함수라, 시그모이드처럼 입력 하나의 곡선으로 그릴 수 없다

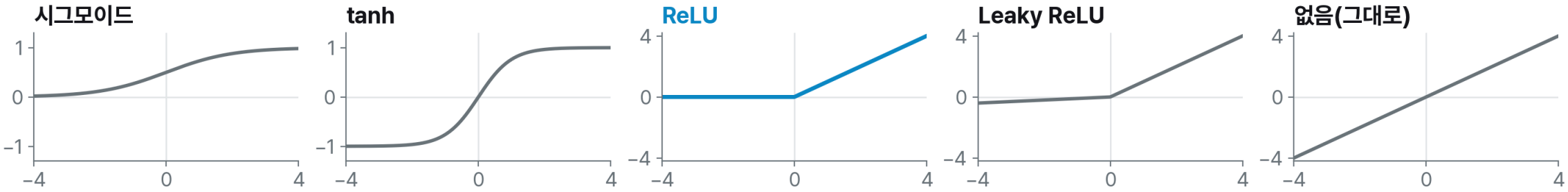


① 점수마다 ② e^z 를 구하고, ③ 그 합 11.213 으로 나누면 확률이 된다

정리: 은닉층은 ReLU, 출력층은 예측할 값의 종류에 따라 활성화 함수를 고른다

함수	출력 범위	가장 큰 기울기	주로 쓰는 층
시그모이드	0～1	0.25	이진 분류 출력층
tanh	−1～1	1	순환 신경망(RNN)의 은닉층
ReLU	**0 이상**	**1**	**은닉층 기본**
Leaky ReLU	실수 전체	1	죽은 ReLU 가 많은 은닉층
소프트맥스	0～1, 합 1	해당 없음	다중 분류 출력층
없음(그대로)	실수 전체	1	회귀 출력층

- 입력 노드 = 특성 수, 출력 노드 = 예측할 값 수
- 파라미터(가중치·편향)는 학습 중에 계산되고, 하이퍼파라미터는 학습 전에 정한다
- 편향은 경계를 원점에서 옮기고, 활성화 함수는 직선을 꺾는다



가로: 가중합 z(−4～4). Leaky ReLU 는 음수 쪽 기울기가 보이도록 α = 0.1 로 그렸다. 소프트맥스는 점수 여러 개를 받는 함수라 곡선으로 그리지 않는다.

6

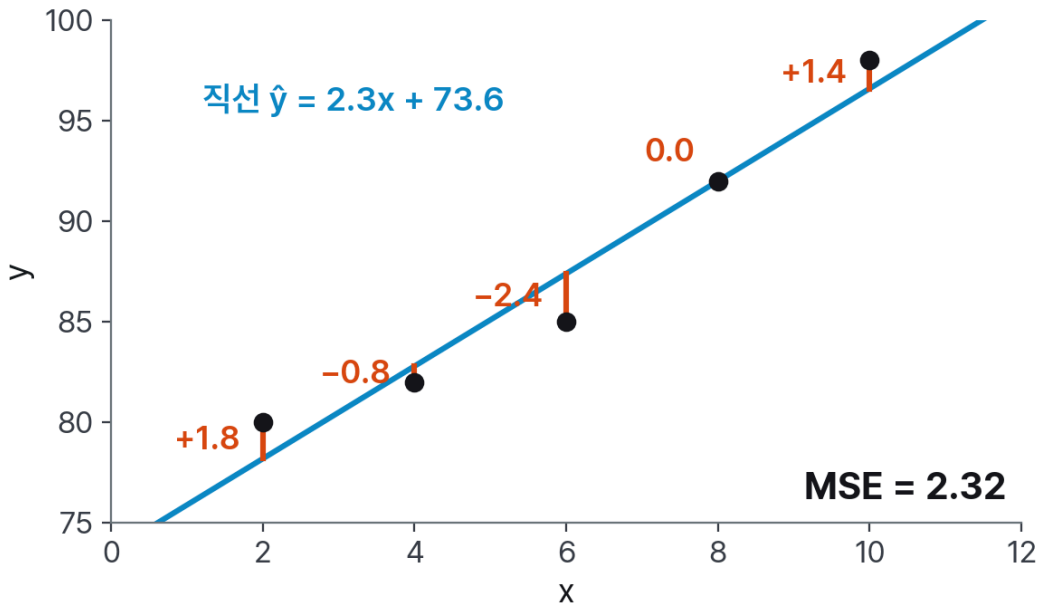
손실 함수

모델이 얼마나 빗나갔는지를 어떤 수 하나로 나타내나?

회귀의 MSE·RMSE·MAE · 분류의 교차 엔트로피 · 채점 지표에 맞춰 손실 함수를 고르는 법

손실 함수는 오차 여러 개를 수 하나로 모으고, 학습할 때는 그 수가 작아지도록 가중치를 바꾼다

- **오차(error)**: 행 하나의 $y - \hat{y}$
  손실(loss): 모든 행의 오차를 모은 수 하나. 「4. 순전파·역전파를 직접 계산하기」의 손실 E(0.2984)도 출력 둘의 오차를 모은 수다
- 예: x = 2, 4, 6, 8, 10 · y = 80, 82, 85, 92, 98. 직선 $\hat{y} = 2.3x + 73.6$ 의 오차는 1.8, −0.8, −2.4, 0.0, 1.4
- 제곱해 평균하면 $(3.24 + 0.64 + 5.76 + 0 + 1.96) \div 5 = 2.32$. 이 직선이 이 값(MSE)을 가장 작게 하는 직선이다

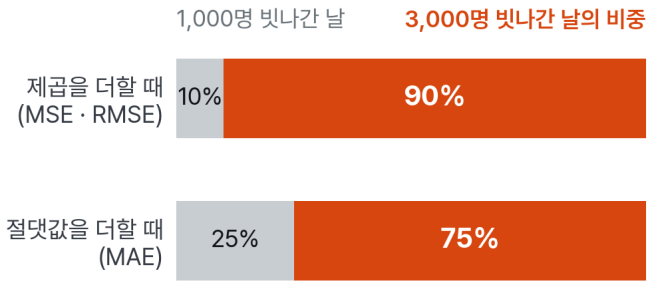


주황 선분: 점마다 오차(실제 값 − 예측 값)

> 손실은 다음 질문에 답한다. **이 가중치로 예측하면 정답에서 평균 얼마나 빗나가는가?**

MSE·RMSE·MAE 는 같은 오차를 다른 방식으로 모으며, 오차가 1,000명·3,000명이면 RMSE 2,236명, MAE 2,000명이다

- 평균 제곱 오차 MSE(mean squared error) $= \frac{1}{n}\sum(y_i - \hat{y}_i)^2$
- 평균 제곱근 오차 RMSE(root mean squared error) $= \sqrt{\mathrm{MSE}}$: 명 단위로 읽을 수 있는 값. 오차를 제곱해 큰 오차가 크게 반영되는 것은 MSE 와 같다
- 평균 절대 오차 MAE(mean absolute error) $= \frac{1}{n}\sum|y_i - \hat{y}_i|$
- 기호: n: 행 수 · y_i: 실제 값 · $\hat{y}_i$: 예측 값



제곱하면 3,000명 빗나간 날이 합의 90%. 그래서 RMSE 가 MAE 보다 크다

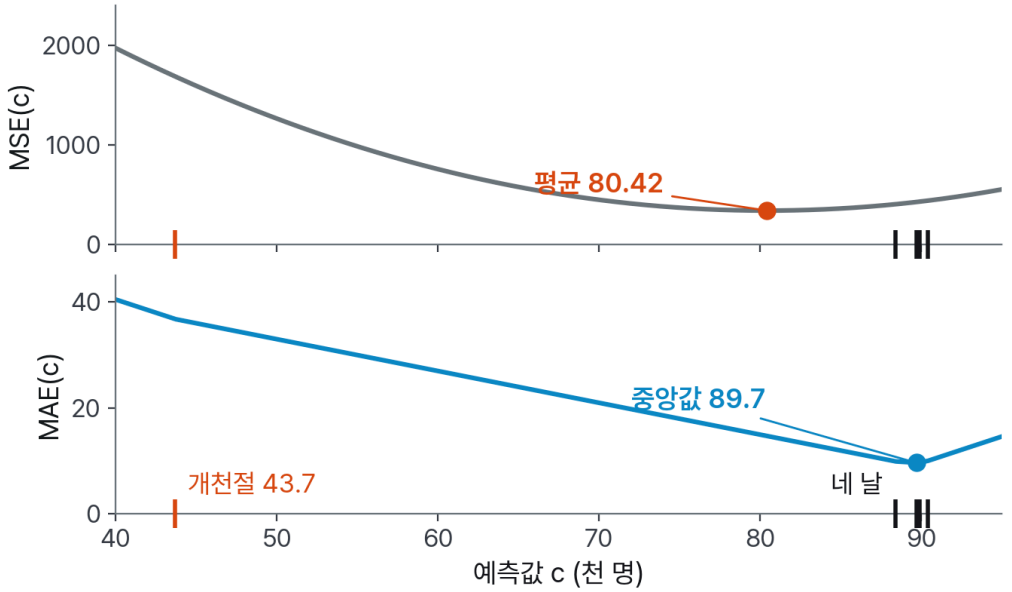
지표	계산 (이틀 오차 1,000명 · 3,000명)	값	읽는 법
MSE	(1,000² + 3,000²) ÷ 2	5,000,000 명²	그대로 읽기 어려움
RMSE	√5,000,000	약 2,236명	약 2,236명 빗나감
MAE	(1,000 + 3,000) ÷ 2	2,000명	하루 평균 2,000명 빗나감

MSE 를 가장 작게 하는 값은 평균, MAE 는 중앙값이라, 드문 날 하나가 있으면 두 값이 80.42 와 89.7 로 달라진다

강남역 다섯 날을 모두 같은 값 c 하나로 예측한다고 가정하고, c 를 바꿔 가며 MSE·MAE 를 계산한다.

날짜(2025)	승차(천 명)	c = 90 의 오차
09-29 (월)	88.4	−1.6
09-30 (화)	90.4	0.4
10-01 (수)	89.9	−0.1
10-02 (목)	89.7	−0.3
10-03 (금, 개천절)	**43.7**	**−46.3**

c = 90 이면 MSE 429.3, MAE 9.74



MSE 를 가장 작게 하는 80.42(다섯 날의 평균)는 개천절 하나 때문에 나머지 네 날보다 8~10천 명 낮다. MAE 를 가장 작게 하는 89.7(다섯 날의 중앙값)은 개천절 승차가 더 낮아도 바뀌지 않는다.

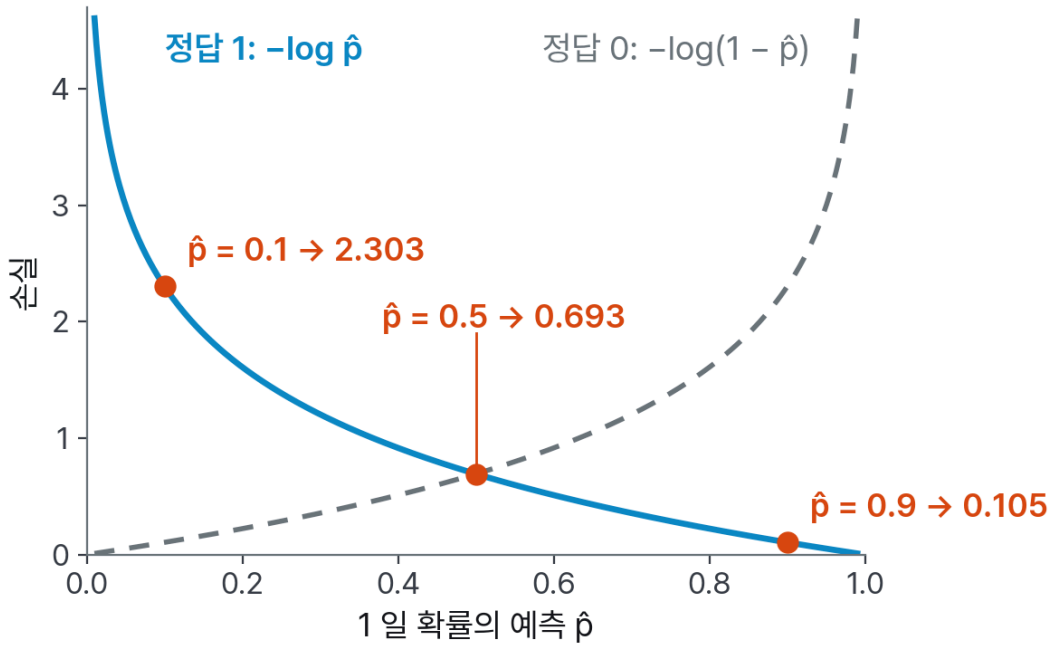
이진 교차 엔트로피는 정답 쪽 확률이 작을수록 손실이 빠르게 커진다

- 이진 교차 엔트로피(binary cross-entropy)는 다음과 같다.

$$L = -\frac{1}{n}\sum_{i}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]$$

  기호: y_i: 정답(0 또는 1) · $\hat{p}_i$: 정답이 1 일 확률의 예측값 · log: 자연로그

- 정답이 1 인 행은 $-\log \hat{p}$ 만 남는다: $\hat{p}$ = 0.9 → 0.105 · 0.5 → 0.693 · 0.1 → 2.303
- 정답 쪽 확률 0.9 를 0.5 로 낮추면 손실이 **6.6배**, 0.1 이면 **21.9배**가 된다



> **틀린 쪽의 예측 확률이 높을수록 손실이 크다.**

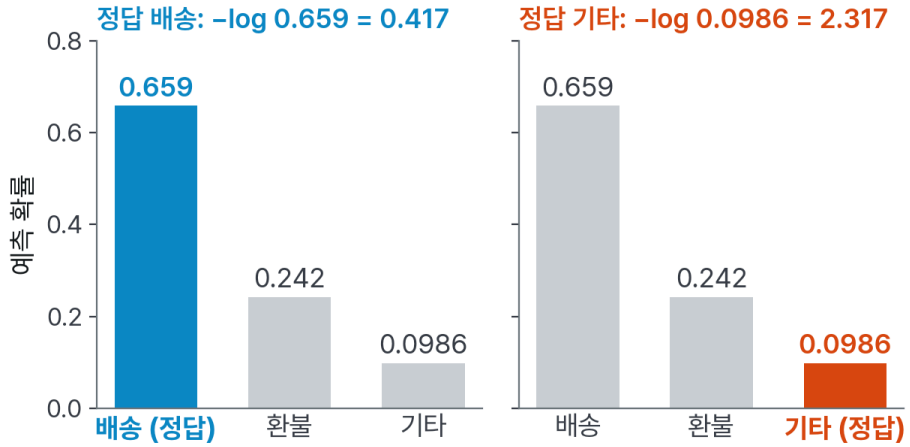
범주형 교차 엔트로피는 정답 범주의 예측 확률 하나로만 계산된다

- 범주형 교차 엔트로피(categorical cross-entropy)는 다음과 같다.

$$L = -\frac{1}{n}\sum_{i}\sum_{c} y_{ic}\log \hat{p}_{ic}$$

기호: y_{ic}: 행 i 의 정답이 범주 c 면 1, 아니면 0 · $\hat{p}_{ic}$: 범주 c 의 예측 확률(소프트맥스 출력)

- 소프트맥스 확률 가운데 정답 범주의 확률 하나만 손실 계산에 쓰인다 (오른쪽 그림)



정답 표기	예 (정답 배송)	PyTorch	Keras	값
원-핫(범주마다 0 또는 1)	[1, 0, 0]	`nn.CrossEntropyLoss`	`categorical_crossentropy`	0.417
범주 번호(sparse)	0	`nn.CrossEntropyLoss` (기본)	`sparse_categorical_crossentropy`	0.417

PyTorch 의 `nn.CrossEntropyLoss` 는 확률이 아니라 소프트맥스 전 점수 z 를 입력으로 받아 함수 안에서 소프트맥스를 계산한다.

글로벌 MLP 의 MASE 는 MAE 손실로 학습했을 때 0.359, MSE 손실로 학습했을 때 0.739 였다

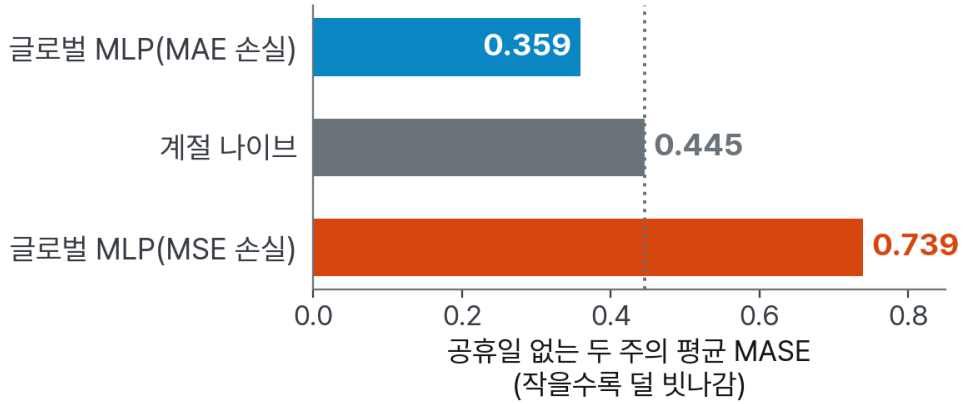
손실은 학습할 때 줄이는 값, **채점 지표**는 학습 뒤 평가표에 적는 값이다. 손실과 채점 지표는 계산 방식이 같은 것끼리 짝짓는다(제곱: MSE ↔ RMSE, 크기: MAE ↔ MAE·MASE). 제곱근은 크기 순서를 바꾸지 않아 MSE 4 와 9 는 RMSE 2 와 3 이 된다.

MASE 는 역마다 MAE 를 정해진 수 Q 로 나눈 값이다. 딥러닝 시계열 과목의 채점 지표이고 식은 다음과 같다.

$$\mathrm{MASE} = \mathrm{MAE} \div Q$$

기호: MAE: 그 역의 채점 7일 평균 절대 오차 · Q: 원점(채점 첫날) 전날까지 계절 나이브(7일 전 값)가 그 역에서 평소 빗나가던 크기, 곧 $|y_t - y_{t-7}|$ 의 평균

Q 는 채점 전에 정해진 수라 MAE 를 줄이면 MASE 도 줄어든다. 예: 잠실역은 Q = 6,405.5명이라 MAE 가 3,203명에서 2,000명으로 줄면 MASE 는 0.500 에서 0.312 로 줄어든다.



서울 지하철 100개 역의 평균. **글로벌 MLP**: 100개 역의 자료를 한 표로 모아 학습한 모델 하나. **공휴일 없는 두 주**: 채점하는 다섯 주 가운데 평일 공휴일이 없는 두 주. **계절 나이브**: 7일 전 값을 그대로 내는 예측

정리: 회귀는 MSE·MAE, 분류는 교차 엔트로피, 고를 때는 채점 지표를 본다

- 손실 = 오차를 수 하나로 모은 것. 학습할 때는 이 수가 작아지도록 가중치를 바꾼다
- MSE 로 학습하면 예측이 평균에, MAE 로 학습하면 중앙값에 가까워진다. 드문 날이 있으면 두 값이 달라진다(강남역 80.42 대 89.7)
- 교차 엔트로피는 정답 쪽 예측 확률이 작을수록 크다(0.9 → 0.105, 0.1 → 2.303)

손실	계산	강남역 다섯 날을 값 하나로 예측할 때 손실을 가장 작게 하는 값	짝이 되는 채점 지표
MSE	오차 제곱의 평균	평균 80.42	RMSE (회귀)
MAE	오차 크기의 평균	**중앙값 89.7**	**MAE · MASE (회귀)**
이진 교차 엔트로피	−log(정답 쪽 확률)의 평균	해당 없음	정확도 · AUC (이진 분류)
범주형 교차 엔트로피	−log(정답 범주 확률)의 평균	해당 없음	정확도 (다중 분류)

7

옵티마이저와 학습 설정

손실이 줄어드는 쪽으로 가중치를 얼마나, 몇 번 바꾸고, 언제 멈추나?

경사하강법과 학습률 · 배치 크기와 에폭 · 모멘텀과 Adam · 과적합·드롭아웃·조기 종료 · 이 과정의 학습 설정

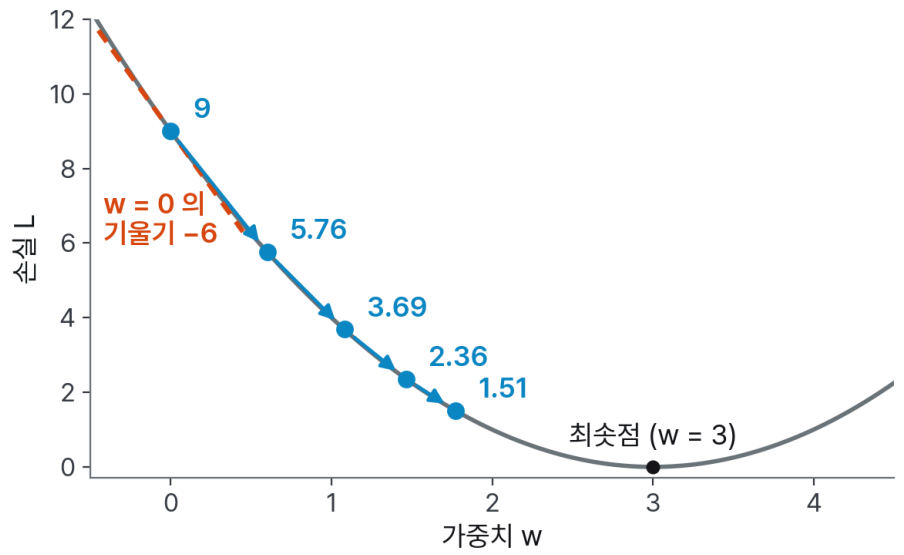
경사하강법은 기울기의 반대쪽으로, 기울기에 학습률을 곱한 만큼 가중치를 옮긴다

- 경사하강법(gradient descent)은 다음 규칙을 되풀이한다.

$$w \leftarrow w - \eta \frac{dL}{dw}$$

기호: w: 가중치 · L: 손실 · $\frac{dL}{dw}$: 기울기 · η: 학습률(learning rate)

- 예: $L(w) = (w-3)^2$, 기울기 $2(w-3)$. $w = 0$ 에서 기울기 −6, $\eta = 0.1 \rightarrow 0 - 0.1 \times (-6) = 0.6$
- 이어서 w 는 0.6 → 1.08 → 1.464, 손실 9 → 5.76 → 3.69 → 2.36. 최솟점에 가까울수록 기울기가 작아 걸음도 작다



경사하강법 한 걸음의 구조는 이렇다.

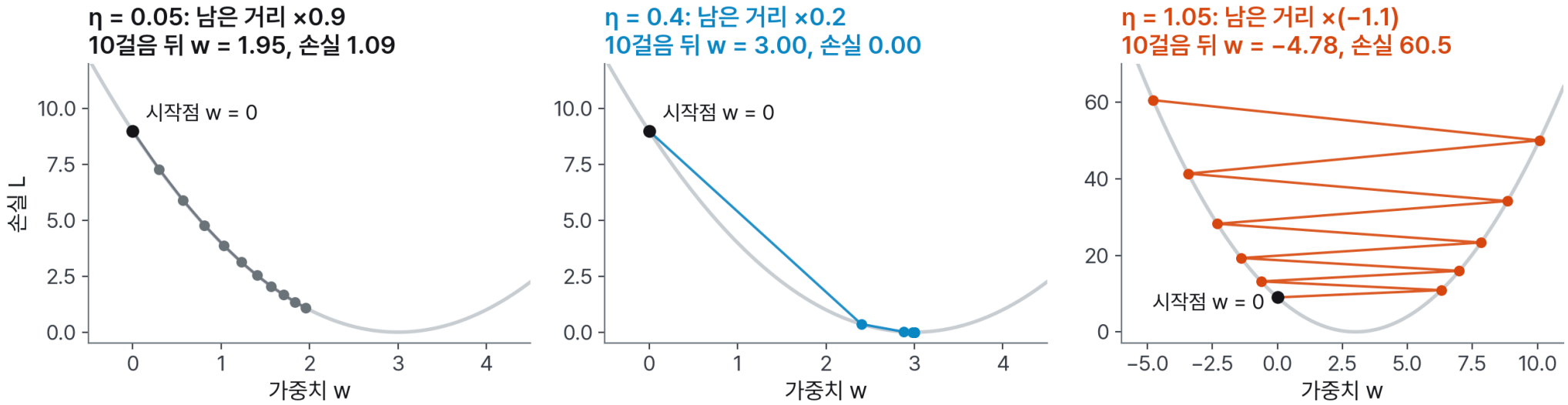
새 가중치 = 지금 가중치 − 학습률 × 기울기

학습률이 작으면 느리고, 알맞으면 몇 걸음에 닿고, 너무 크면 최솟점에서 점점 멀어진다

학습률 η 는 한 걸음의 거리가 아니라 기울기에 곱하는 수다(걸음 거리 = 학습률 × 기울기 크기). 같은 $L(w) = (w-3)^2$ 를 $w = 0$ 에서 10걸음 가면, 기울기 $2(w-3)$ 때문에 한 걸음 뒤 최솟점까지 남은 거리는 이렇다.

$$w^{+} - 3 = (w-3) - \eta \cdot 2(w-3) = (1-2\eta)(w-3)$$

η = 0.05 → ×0.9 · η = 0.4 → ×0.2 · η = 1.05 → ×(−1.1). $|1-2\eta| > 1$ 이면 남은 거리가 걸음마다 커진다: **발산**(divergence)



에폭은 학습 자료 전체를 한 번 사용하는 것이고, 배치 크기는 가중치를 한 번 고칠 때 사용하는 행 수다

학습 자료 2,000행 · 배치 크기 200



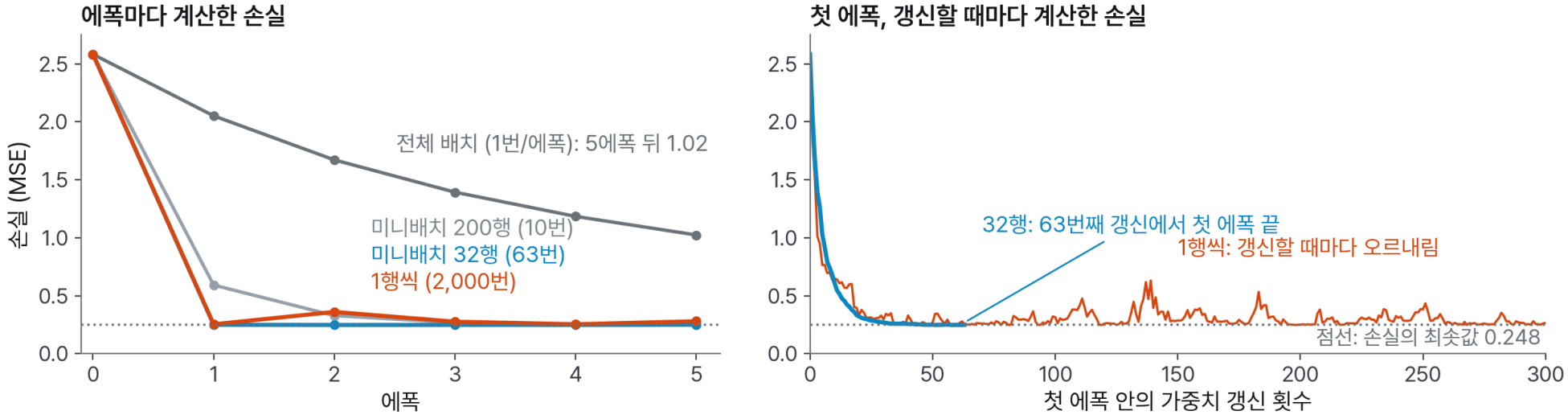
- **에폭(epoch)**: 학습 자료 전체를 한 번 사용하는 것. **배치 크기(batch size)**: 가중치를 한 번 고칠 때 사용하는 행 수
- 배치 크기 32 → 2,000 ÷ 32 = 62.5 → 63번(마지막 배치는 16행)
- 16·32·64·128·256 처럼 2의 거듭제곱을 흔히 쓴다. 이 과정의 MLP 는 256 이다

배치 크기	한 에폭의 갱신 횟수
2,000 (전체)	1
200	10
32	63
1	2,000

배치가 작을수록 가중치를 자주 고치지만, 손실이 더 크게 오르내린다

1에폭 뒤 손실: 전체 배치 2.05 · 미니배치 200행 0.59 · 32행 0.25 · 1행씩 0.25. 5에폭 뒤 전체 배치는 1.02(갱신 5번뿐), 1행씩은 에폭마다 0.25~0.36 을 오간다.

- **미니배치**(mini-batch): 전체 행 가운데 일부(예: 200행)를 묶은 것. 한 번 고칠 때 이 행들로 기울기를 구한다
- **확률적 경사하강법**(SGD, stochastic gradient descent): 좁게는 1행씩, 넓게는 무작위로 뽑은 미니배치로 고치는 방법. PyTorch 의 SGD 는 넓은 뜻



자료: $y = 2x + 1$ 에 잡음을 더한 2,000행, 학습률 0.1, $w = b = 0$ 에서 출발(손실 2.58). 점선 0.248: 잡음 때문에 더 내려갈 수 없는 손실. 그림의 「1행씩 SGD」는 좁은 뜻이다.

옵티마이저는 「몇 행으로 기울기를 구하나」와 「기울기로 걸음을 어떻게 정하나」, 두 기준으로 나뉜다

옵티마이저(optimizer)는 기울기로 가중치를 고치는 규칙이다. 자주 쓰는 옵티마이저 10가지(GD 와 BGD 는 같은 방법의 두 이름)는 아래 격자의 행 셋·열 넷에 들어가고, 이 과정이 쓰는 AdamW 를 다섯째 열에 더했다.

한 번에 사용하는 행 ↓ \ 걸음 규칙: 기울기로 걸음을 정하는 법 →	기본 기울기 그대로	+ 지난 걸음 이어 받기	+ 파라미터마다 걸음 크기 맞추기	앞의 두 가지 합치기	Adam + 가중치 감쇠
전체 배치 모든 행	**GD = BGD**	잘 안 씀	잘 안 씀	잘 안 씀	잘 안 씀
미니배치 일부 행	**MBGD**	**모멘텀 · NAG**	**Adagrad · RMSProp · AdaDelta**	**Adam · NAdam**	**AdamW** 이 과정: 256행
1행씩 행 하나	**SGD(좁은 뜻)**	잘 안 씀	잘 안 씀	잘 안 씀	잘 안 씀

- **MBGD**: 미니배치 경사하강법(mini-batch gradient descent). PyTorch 의 SGD 는 받은 미니배치로 고치는 넓은 뜻이라 MBGD 와 같다
- 「잘 안 씀」: 규칙을 적용할 수는 있지만 딥러닝에서 드문 조합. NAG·Adagrad·RMSProp·AdaDelta·NAdam 은 이 문서에서 다루지 않는다

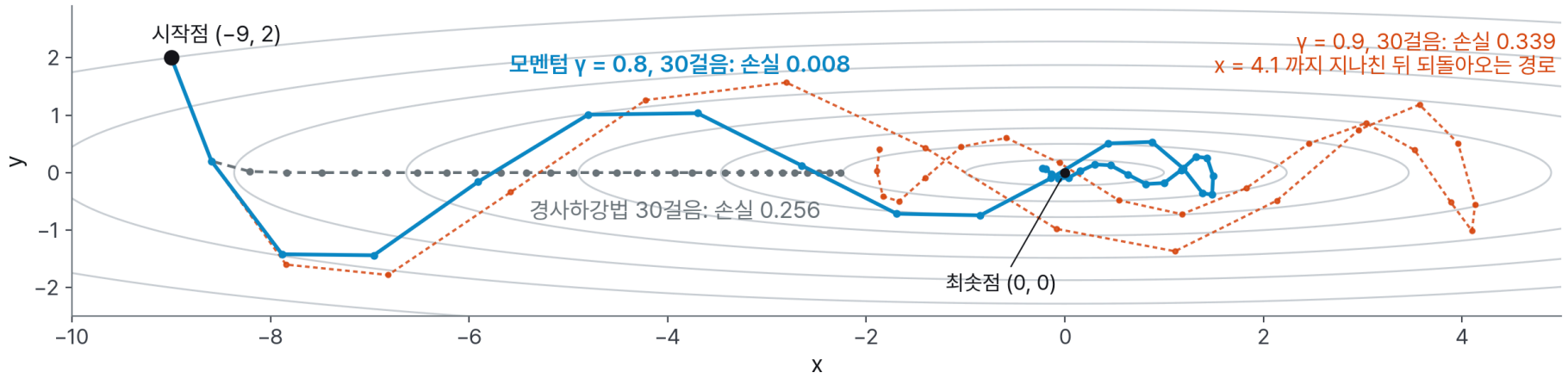
모멘텀(γ = 0.8)은 지난 걸음을 일정 비율 이어 받아, 한쪽만 가파른 손실에서 30걸음 뒤 손실을 약 32분의 1 로 줄였다

모멘텀(momentum)은 다음과 같다.

$$v_t = \gamma v_{t-1} + \eta\, g_t, \qquad w \leftarrow w - v_t$$

기호: g_t: 이번 기울기 · v_t: 이번 걸음 · γ: 지난 걸음을 남기는 비율(흔히 0.9)

- 기울기가 계속 1, $\eta = 0.1$, $\gamma = 0.9$ 면 걸음이 0.1 → 0.19 → 0.271 → … → 1.0 에 다가간다(경사하강법 걸음의 10배)
- 아래 손실 $x^2/20 + y^2$ 는 최솟점에서 같은 거리만큼 떨어지면 y 방향 기울기가 x 방향의 20배다($x = y = 1$ 에서 0.1 대 2). (−9, 2) 에서 같은 $\eta = 0.45$, $\gamma = 0.8$ 로 30걸음



γ 를 흔히 쓰는 0.9 로 키우면 30걸음 뒤 손실이 0.339 로 경사하강법(0.256)보다 크다. 너무 크게 이어 받으면 최솟점을 여러 번 지나쳐 손실이 늦게 줄어든다(주황 점선).

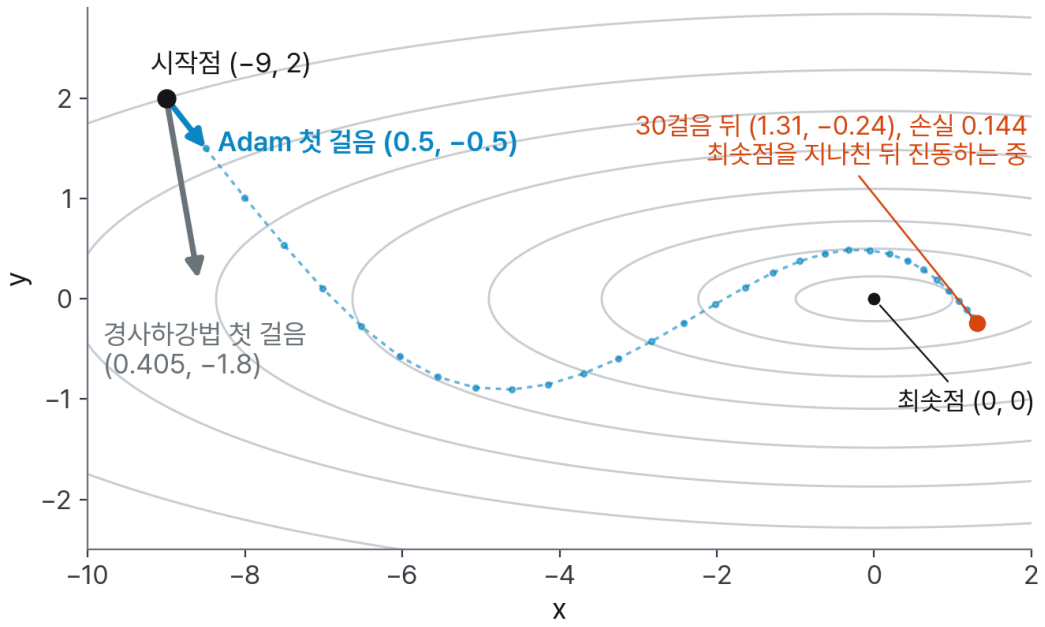
Adam 은 기울기를 파라미터마다 그 크기로 나눠, 완만한 방향도 같은 걸음으로 옮긴다

Adam(adaptive moment estimation, Kingma·Ba 2014)은 다음과 같다.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$w \leftarrow w - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

기호: g_t: 이번 기울기 · m: 기울기의 이동 평균 · v: 기울기 제곱의 이동 평균 · $\hat{m}, \hat{v}$: 처음 몇 걸음의 치우침을 고친 값 · ϵ: 0 으로 나누지 않게 더하는 작은 수. 여기의 v 는 앞 장 모멘텀의 v(이번 걸음)와 다른 뜻

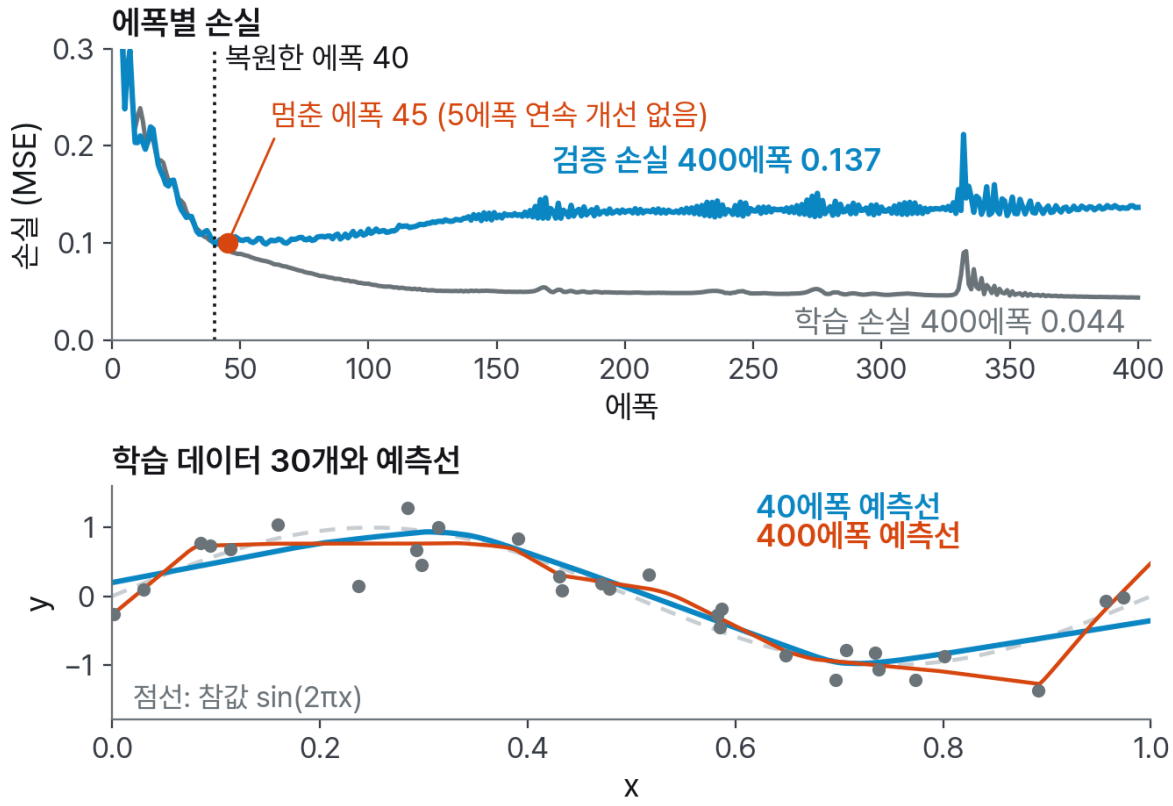
기본값 $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ (논문과 PyTorch 가 같음)



앞 장 출발점의 기울기 (−0.9, 4.0). 경사하강법(η = 0.45) 첫 걸음은 기울기에 비례해 거의 아래로, Adam 첫 걸음은 기울기 크기와 상관없이 방향마다 η 다. η = 0.5 는 걸음이 그림에서 보이게 키운 값이다. 30걸음 뒤 손실은 Adam 0.144, 경사하강법 0.256, 모멘텀(γ = 0.8) 0.008.

학습 손실이 줄어도 검증 손실이 다시 커지면 과적합이고, 조기 종료는 그 앞에서 멈춘다

- **과적합(overfitting)**: 학습 자료에만 맞고 새 자료에는 덜 맞는 상태. 학습에 쓰지 않은 **검증 자료(validation)**의 손실로 알아본다
- 예(점 30개, 은닉 128·128 MLP, Adam): 40에폭 학습 0.101·검증 0.100 → 400에폭 학습 0.044·검증 0.137
- **조기 종료(early stopping)**: 검증 손실이 5에폭 이어서 나아지지 않으면 멈추고 가장 좋았던 에폭의 가중치를 복원한다. 이 예는 45에폭에서 멈추고 40에폭의 가중치를 복원했다
- 이 과정의 MLP 도 예측 첫날 직전 56일을 검증 구간으로 떼어 같은 5에폭 기준으로 멈춘다

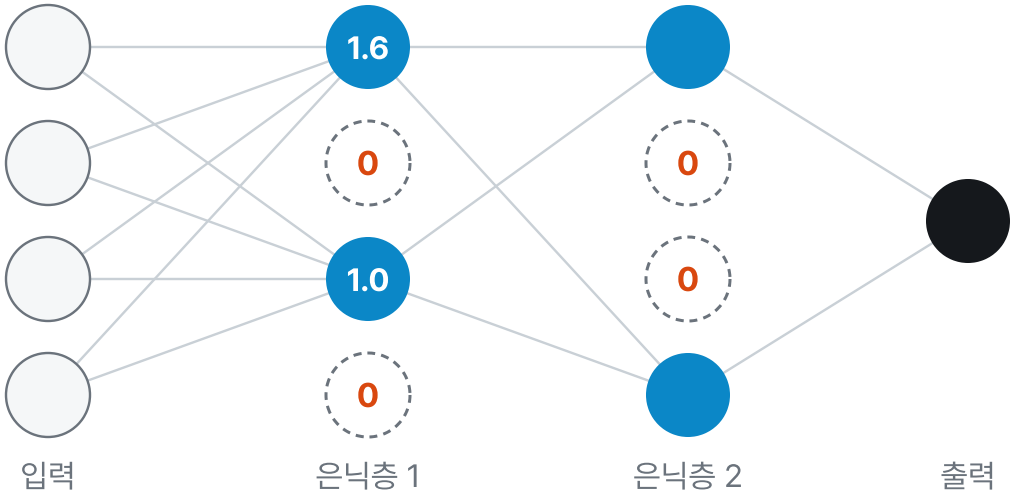


조기 종료는 가장 낮은 점을 보장하지 않는다. 이 예의 최저는 60에폭 0.099

드롭아웃은 학습할 때만 노드 출력을 확률 p 로 0 으로 만들고, 예측할 때는 모두 쓴다

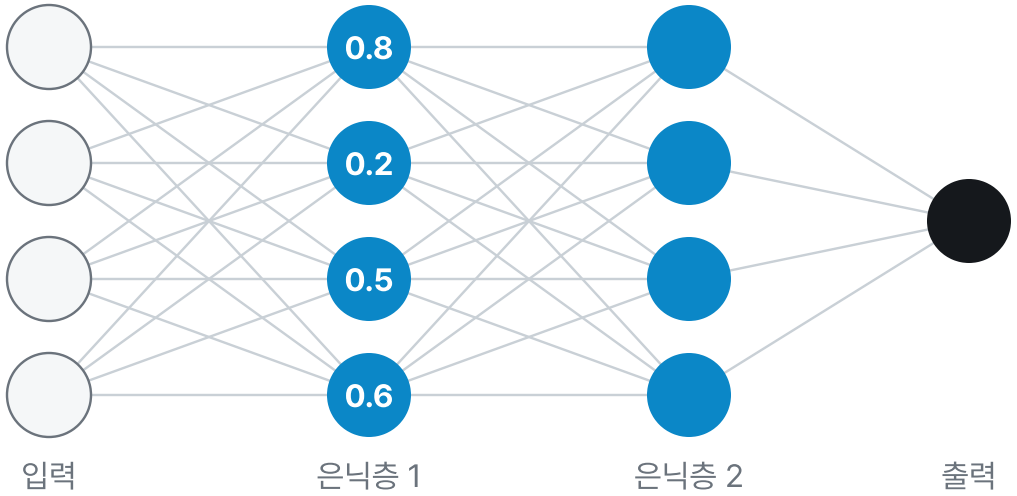
학습할 때 (p = 0.5)

끈 노드는 0, 남긴 노드는 값 × 1/(1 − p) = ×2



예측할 때

끄지 않고 값을 그대로

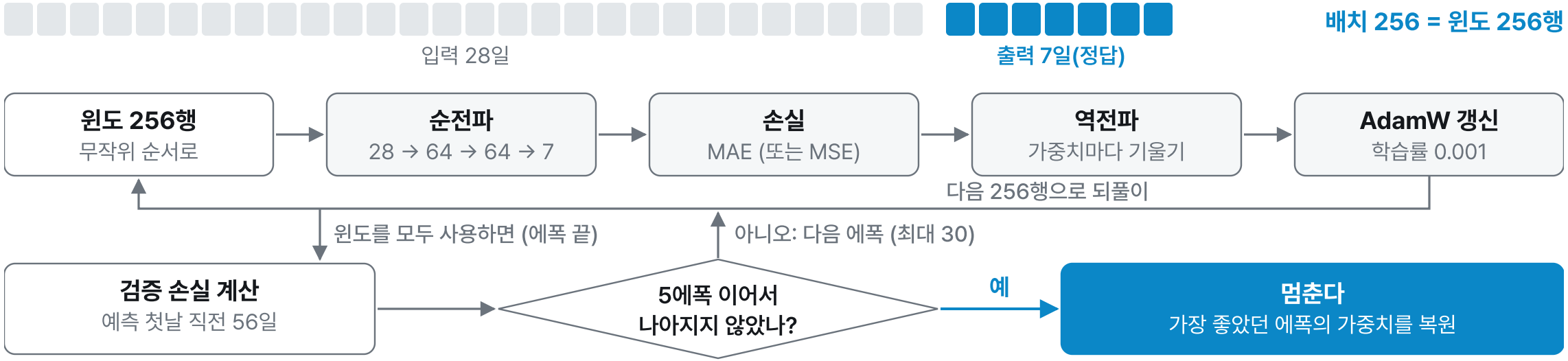


- **드롭아웃(dropout)**: 가중치를 고칠 때마다 은닉 노드를 무작위로 골라 출력을 0 으로 만든다. 가중치가 특정 노드 몇 개의 출력에만 맞춰지지 않아 과적합이 준다
- 예: 출력 (0.8, 0.2, 0.5, 0.6), 이번 선택 (남김, 끔, 남김, 끔) → (1.6, 0, 1.0, 0). 2배는 출력의 평균 크기를 예측 때와 맞추려는 것(PyTorch `nn.Dropout`, 기본 p = 0.5)

Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR 15, 1929–1958.

이 과정의 글로벌 MLP 는 AdamW·학습률 0.001·배치 256·최대 30에폭·조기 종료 5에폭으로 학습한다

윈도(window) 한 행: 한 역의 연속한 28일(입력)과 바로 뒤 7일(출력)을 한 행으로 자른 것



- **AdamW**(Loshchilov·Hutter 2017): Adam 에 가중치를 걸음마다 조금씩 0 쪽으로 줄이는 **가중치 감쇠(weight decay)**를 따로 더한 것. 이 과정은 감쇠 0.01
- **시드(seed)** 0: 가중치 초깃값과 윈도 순서를 정하는 난수의 출발값

정리: 가중치는 배치마다 학습률 × 기울기만큼 고치고, 검증 손실을 보고 멈춘다

- 경사하강법 $w \leftarrow w - \eta \frac{dL}{dw}$. 학습률이 작으면 느리고 너무 크면 발산한다(0.05 · 0.4 · 1.05)
- 배치 크기는 가중치를 한 번 고칠 때 사용하는 행 수, 에폭은 자료 전체를 한 번 사용하는 것. 작은 배치는 가중치를 자주 고치고, 손실이 더 크게 오르내린다
- 모멘텀은 지난 걸음을 이어 받고, Adam 은 파라미터마다 걸음 크기를 맞춘다. 과적합은 검증 손실로 보고 드롭아웃·조기 종료로 줄인다

설정	정하는 것	이 과정의 글로벌 MLP
옵티마이저 · 학습률	걸음 규칙, 기울기에 곱하는 수	**AdamW · 0.001 (감쇠 0.01)**
배치 크기 · 에폭	한 번 고칠 때 사용하는 행 수, 자료 전체를 몇 번 사용하나	**256 · 최대 30**
조기 종료	언제 멈추나	**검증 56일의 손실이 5에폭 이어서 나아지지 않으면 멈춤**

8

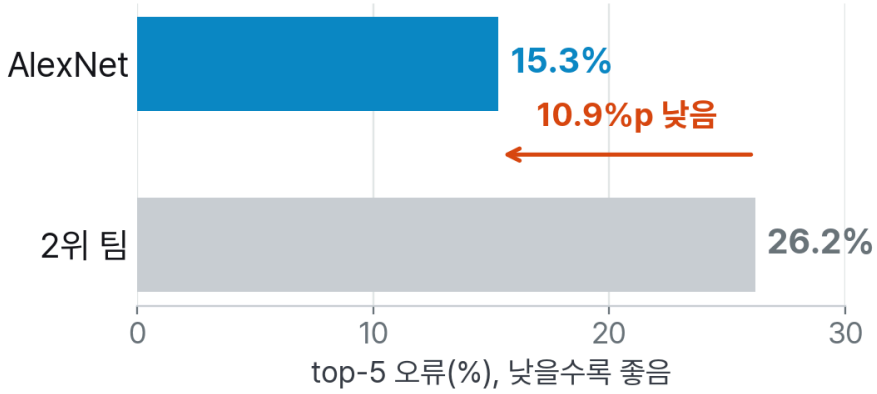
오늘날의 딥러닝

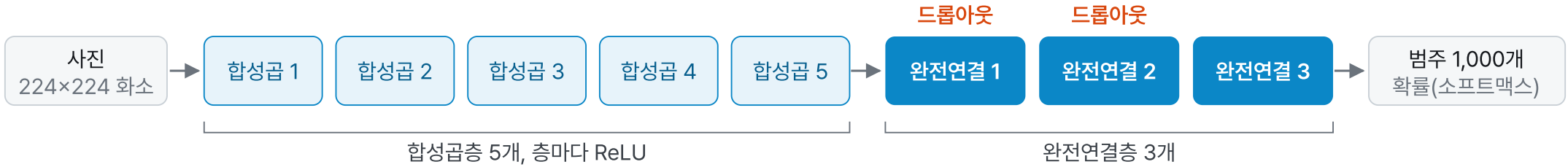
2012년 뒤 무엇이 바뀌었고, 이 과정에서는 어떤 모델을 쓰나?

AlexNet 과 GPU · 더 깊은 신경망을 학습하게 한 방법들 · 트랜스포머와 사전학습 모델 · 시계열의 Chronos-2

2012년 AlexNet 은 GPU 2장으로 학습해 이미지넷 대회 top-5 오류 15.3% 로, 2위 팀(26.2%)보다 10.9%p 낮았다

- **이미지넷 대회(ILSVRC-2012)**: 사진 한 장의 범주를 1,000개 가운데 고른다. **top-5 오류**는 모델이 낸 후보 5개 안에 정답이 없는 사진의 비율
- **GPU(그래픽 처리 장치)**: 같은 종류의 곱셈·덧셈을 한꺼번에 많이 하는 칩이라 가중합 계산이 빠르다. GTX 580(3GB) 두 장으로 5~6일 학습
- 앞에서 본 두 방법이 쓰였다: **ReLU**(4층 합성곱망이 CIFAR-10 학습 오류 25% 에 이르는 데 tanh 보다 약 6배 빨랐다), 앞 두 완전연결층의 **드롭아웃**





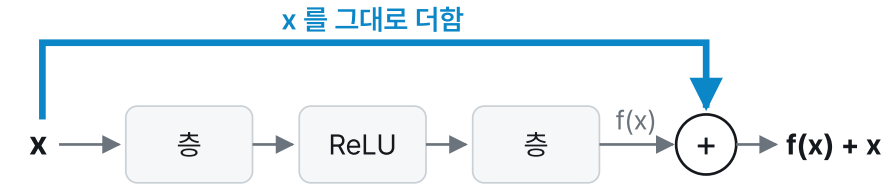
AlexNet 의 층 구성. 파라미터(가중치·편향)는 모두 6천만 개. 합성곱층(convolutional layer)은 사진의 작은 영역(첫 층은 11×11 화소)을 옮겨 가며 같은 가중치를 곱해 더하는 층이다. 완전연결층(fully connected layer)은 앞 층의 모든 노드와 이어진 층으로, 앞에서 본 MLP 의 은닉층이 이것이다.

Krizhevsky, Sutskever, Hinton(2012), ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS. 15.3% 는 신경망 7개 예측의 평균.

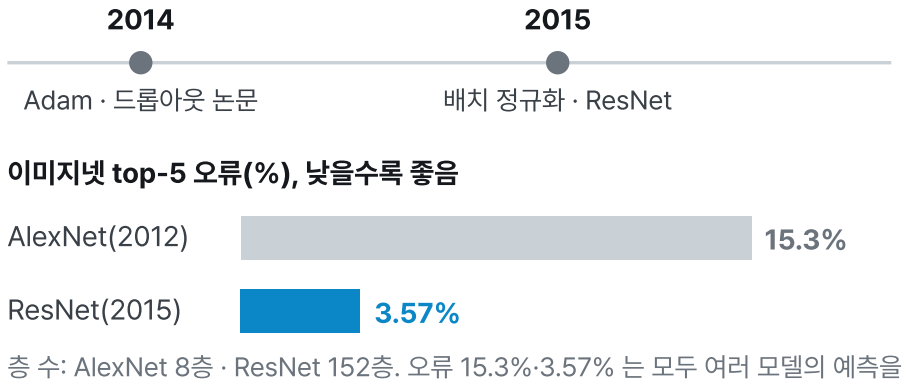
2014~2015년 Adam·배치 정규화·ResNet 이 나와 더 깊은 신경망을 학습하게 됐다

- 2014년 Adam(Kingma·Ba)과 드롭아웃을 정리한 논문(Srivastava 외, JMLR)이 나왔다. 둘 다 「7. 옵티마이저와 학습 설정」에서 다뤘고, 드롭아웃은 2012년 AlexNet 이 이미 썼다
- **배치 정규화(batch normalization, 2015, Ioffe·Szegedy)**: 층의 입력값을 미니배치마다 평균 0·분산 1 로 맞춘다(예: 2·4·6 → −1.22·0·1.22). 논문의 이미지 분류에서 같은 정확도에 이르는 학습 단계 수가 14분의 1
- **ResNet(2015, He 외)**: 층의 출력에 입력을 더하는 연결 $f(x) + x$ 로 152층까지 학습했다. 미분하면 $f'(x) + 1$ 이라, 층마다 곱해지는 기울기가 0 근처로 줄지 않는다

ResNet 블록: 층 둘을 건너뛰어 입력 x 를 더한다



기울기(미분): f′(x) 가 0.1 이어도 f′(x) + 1 = 1.1
5층을 지나며 곱하면 $0.1^5 = 0.00001$ 대 $1.1^5 \approx 1.61$



층 수: AlexNet 8층 · ResNet 152층. 오류 15.3%·3.57% 는 모두 여러 모델의 예측을 평균한 값

Srivastava 외(2014) JMLR 15 · Kingma, Ba(2014) arXiv 1412.6980 · Ioffe, Szegedy(2015) arXiv 1502.03167 · He 외(2015) arXiv 1512.03385

2017년 트랜스포머는 어텐션으로, 낱말마다 자기를 포함한 모든 낱말의 값을 유사도에 따른 비율로 가중평균한다



- **트랜스포머**(transformer, 2017, Vaswani 외): 어텐션만으로 만든 신경망. 그전의 순환 신경망(RNN)은 **순환 구조**, 곧 앞 낱말의 계산이 끝나야 다음 낱말을 계산하는 구조였다
- **어텐션**(attention): 낱말마다 자기를 포함한 각 낱말과 얼마나 비슷한지를 점수(유사도)로 계산하고, 소프트맥스로 점수를 합이 1 인 비율로 바꿔 그 낱말들의 값을 가중평균한다
- 모든 낱말의 어텐션을 한꺼번에 계산할 수 있어 GPU 로 빠르게 학습한다

낱말 1 의 새 값 구하기: 세 낱말(자기 포함)의 값을 사용

낱말	유사도	소프트맥스 비율	값	비율 × 값
낱말 1	2	0.665	× 10	6.65
낱말 2	1	0.245	× 20	4.90
낱말 3	0	0.090	× 30	2.70

점수 (2, 1, 0)
→ 소프트맥스 → 비율

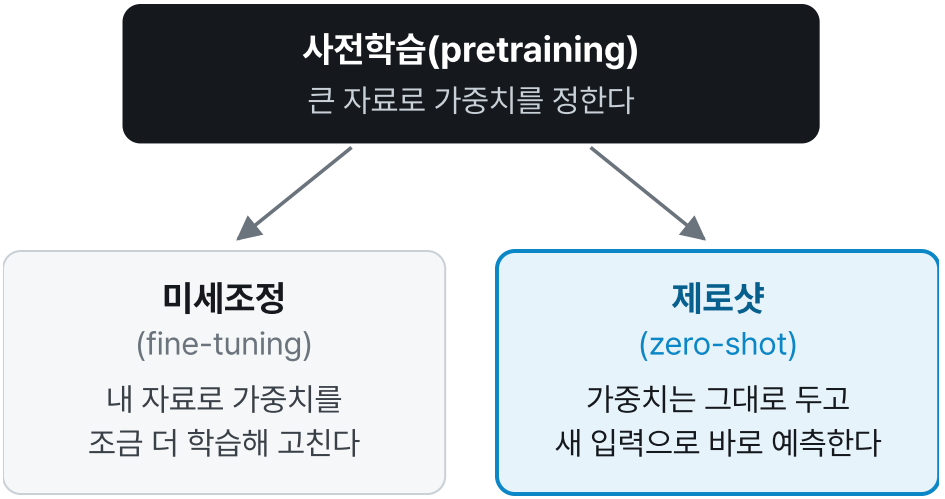
더하면 6.65 + 4.90 + 2.70 = 14.25
이 14.25 가 낱말 1 의 새 값(가중평균)

수 하나로 줄인 예. 실제로는 점수를 두 낱말의 수 벡터로 구하고(내적), 값도 수 여러 개다.

Vaswani 외(2017), Attention Is All You Need, arXiv 1706.03762

2018년 뒤로는 큰 자료로 미리 학습한 모델을 가져다 쓰는 방식이 표준이 됐다

- 2018년 GPT(OpenAI, 6월)·BERT(구글, 10월): 트랜스포머를 큰 글 자료로 미리 학습한 모델(**사전학습 모델**, pretrained model)을 작업마다 미세조정해 쓴다
- 2022년 11월 30일 ChatGPT 공개 뒤, 글·그림을 만들어 내는 생성형 AI 가 널리 쓰인다
- 시계열에도 2024년 Chronos, 2025년 Chronos-2 같은 사전학습 모델이 나왔다(다음 장)
- 앞에서 본 층별 사전학습(학습 전에 가중치 초깃값을 정하는 방법)과는 다른 방식이다



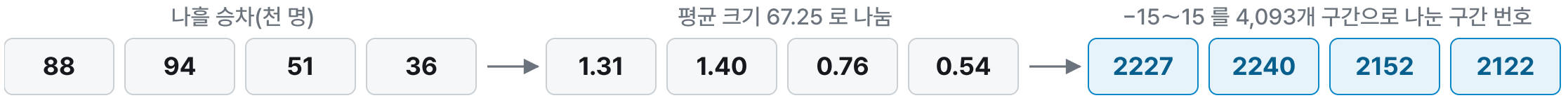
사전학습 모델을 쓰는 두 가지 방법. 이 과정의 Chronos-2 는 제로샷으로 쓴다.

OpenAI(2018-06-11) Improving language understanding with unsupervised learning · Devlin 외(2018) arXiv 1810.04805 · OpenAI(2022-11-30) Introducing ChatGPT

Chronos 는 계열 값을 토큰 번호로 바꿔 언어 모델처럼 사전학습하고, Chronos-2 는 연속한 16일의 값(패치, patch)을 토큰 하나로 쓴다

- **토큰**(token): 모델 입력의 한 단위. 언어 모델은 낱말(또는 낱말을 더 잘게 나눈 서브워드)을 토큰 번호로 바꿔 입력한다
- **Chronos**(Ansari 외, 2024)는 값을 평균 크기로 나눈 뒤 구간 번호로 바꿔 언어 모델 구조(T5)로 사전학습했다. **Chronos-2** 는 여러 계열과 **공변량**(함께 입력하는 다른 열, 예: 공휴일)도 받을 수 있다
- 이 과정은 Chronos-2 를 제로샷으로, 원점 전날까지 512일의 승차 한 열만 입력해 쓴다(공변량은 입력하지 않음)

Chronos(2024): 값 하나 → 토큰 하나



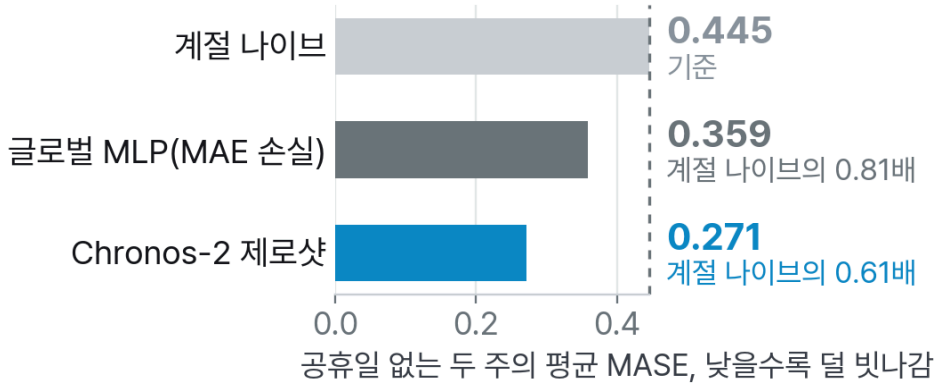
Chronos-2(2025): 16일 패치 하나 → 토큰 하나



위쪽 예의 네 값은 설명용이고, 구간 번호는 Chronos 공개 코드와 같은 방식(구간 너비 약 0.0073)으로 계산했다.

Ansari 외(2024) arXiv 2403.07815 · Ansari 외(2025) arXiv 2510.15821 · Hugging Face amazon/chronos-t5 · amazon/chronos-2 의 config.json(input_patch_size 16)

시계열에도 사전학습 모델이 나왔고, 이 과정은 처음부터 학습한 글로벌 MLP 와 제로샷 Chronos-2 를 같은 MASE 로 비교한다



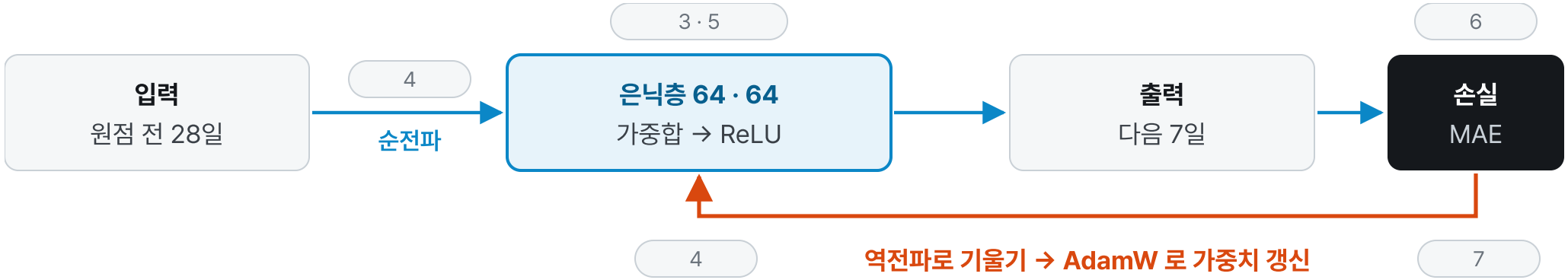
MASE = 모델의 평균 절대 오차 ÷ 계절 나이브(7일 전 같은 요일 값)가 원점(채점 첫날) 전날까지 평소 빗나가던 크기. 1 보다 작으면 평소보다 덜 빗나간다.

구분	글로벌 MLP	Chronos-2
구조	28 → 64 → 64 → 7, ReLU	트랜스포머
가중치	6,471개를 100개 역 자료로 처음부터 학습(AdamW, MAE 손실)	**사전학습한 값 그대로(제로샷)**
입력	원점 전 28일	원점 전날까지 512일, 승차 한 열(공변량 없음)
출력	7일 값	7일 분위수 9개(이 과정이 0.1~0.9 를 요청), 0.5 분위수를 예측값으로

공휴일 있는 세 주에는 셋 다 2.7 안팎(계절 나이브 2.767 · 글로벌 MLP 2.717 · Chronos-2 2.763)이라 어느 모델도 계절 나이브보다 낫다고 말할 수 없다. 두 모델 모두 예측할 날이 공휴일인지를 입력으로 받지 않았다.

Ansari 외(2024) arXiv 2403.07815 · Ansari 외(2025) arXiv 2510.15821 · 수: 서울 지하철 100개 역, 딥러닝 시계열 1·2편 평가표

정리: 신경망은 층마다 가중합과 활성화를 계산하고, 손실의 기울기로 가중치를 고친다



딥러닝 시계열 1편의 글로벌 MLP 학습 단계 한 번. 회색 상자 안 숫자는 그 내용을 다룬 목차 번호.

- **계산**: 층마다 가중합 → 활성화 함수(순전파). 은닉층이 있어야 XOR 같은 문제가 풀린다
- **학습**: 손실(MSE·MAE·교차 엔트로피)의 기울기를 역전파로 구하고, 옵티마이저(경사하강법·Adam 등)가 학습률에 맞춰 가중치를 고친다. 검증 손실이 더 줄지 않으면 멈춘다
- **오늘날**: 2012년 GPU·ReLU·드롭아웃으로 깊은 신경망이 쓰이기 시작했고, 2017년 트랜스포머 뒤로 사전학습 모델(Chronos-2 등)을 가져다 쓴다

> **다음, 딥러닝 시계열 1편 「글로벌 모델: 지하철 100개 역을 신경망 하나로」**: 이 신경망으로 역 100개의 다음 7일 승차를 예측한다